

Nazwa Kodowa Projektu: Delfinek

Pełna nazwa projektu: System zarządzający szkołą pływania
"Delfinek"

Zespół:

Łukasz Deja (team leader)
Natalia Parszywka
Miłosz Błaszczuk

Repozytorium GitHub:

<https://github.com/lukaszdeja/Delfinek>

Projekt na GitHubie:

<https://github.com/users/lukaszdeja/projects/1>

Tablica kanbanowa projektu (Trello):

<https://trello.com/b/H3ZY2HrB/projekt-delfinek>

23 czerwca 2026

1 Co realizuje projekt?

Projekt ma realizować system zarządzania szkołą pływania **Delfinek**, który organizuje lekcje pływania dla dzieci i dorosłych, wspiera zapisy na zajęcia, przydziela instruktorów i torów na basenie, ustala grafik zajęć i informuje kursantów o zmianach w grafiku. Celem projektu jest usprawnienie działania szkoły pływania i ułatwienie zarządzania nią, a także przyciągnięcie wielu nowych klientów do szkoły pływania poprzez reklamę i ułatwienie zapisywania się na kurs, co skutkować będzie większymi zyskami.

2 Wizja rozwiązania

System zostanie zrealizowany za pomocą aplikacji webowej, najprawdopodobniej strony internetowej, z której korzystać będą mogli klienci uczestniczący w zajęciach oraz instruktorzy a także administrator systemu. Te same funkcjonalności będzie realizować aplikacja mobilna stworzona dla wygody użytkowników.

2.1 Product Owner

Product Ownerem jest właściciel szkoły pływania, menedżer systemu, który jest w kontakcie z właścicielem obiektu sportowego - basenu i posiada informacje dotyczące obiektu. To on jest źródłem wiedzy dla systemu oraz dla zespołu projektowego odnośnie wymagań samego w sobie projektu. Wywiad z nim będzie źródłem informacji.

2.2 Podstawowe wymagania użytkownika

Przeprowadzono wywiad z osobami, które będą pełnić role pracowników biura, instruktorów oraz klientów aby sformułować wymagania.

Klient szkoły pływania:

- Jako uczestnik chcę zapisać się na zajęcia w dogodnym dla siebie terminie.
- Jako uczestnik chcę być powiadamianym o zmianach w harmonogramie.
- Jako uczestnik chcę być powiadamianym o pracach remontowych w obiekcie sportowym i ograniczeniach związanych z obiektem.
- Jako uczestnik chcę być powiadamianym o cenach kursu oraz ich zmianach.
- Jako uczestnik chcę mieć możliwość komunikacji z instruktorem i administratorem systemu.

Instruktor:

- Jako instruktor chcę mieć wgląd do grafiku, aby wiedzieć kiedy prowadzę lekcje.
- Jako instruktor chcę mieć wgląd do danych użytkowników moich lekcji, aby móc odpowiednio dostosować zajęcia do uczestników.
- Jako instruktor chcę być powiadamianym o zmianach w harmonogramie.
- Jako instruktor chcę być powiadamianym o moich nowych kursantach.

Administrator systemu:

- Jako pracownik biura chcę móc zarządzać harmonogramem uczestników, aby uniknąć konfliktów i zbyt dużej liczby kursantów w danych godzinach.
- Jako pracownik biura chcę mieć informacje o obiekcie sportowym w danych godzinach i dostępnych torach oraz o pracach remontowych.
- Jako pracownik biura chcę móc zarządzać torami basenowymi w przeznaczonych dla szkoły godzinach.
- Jako pracownik biura chcę mieć informacje o preferencjach godzinowych klientów, aby móc efektywnie dopasować grafik.
- Jako pracownik biura chcę powiadamiać kursantów oraz instruktorów o zmianach w harmonogramie oraz cenach za kursy.
- Jako pracownik biura chcę przekazywać instruktorom informacje, które nie są poufne i nie naruszają prywatności klientów.

3 Zakres usług systemu

System obejmuje następujące funkcjonalności:

- Rejestracja i logowanie użytkowników (klienci, instruktorzy, administrator).
- Zapisy uczestników na zajęcia pływania z uwzględnieniem dostępnych terminów.
- Zarządzanie harmonogramem zajęć (tworzenie, edycja, usuwanie).
- Przydzielanie instruktorów oraz torów basenowych do konkretnych zajęć.
- Powiadamianie użytkowników (klientów i instruktorów) o zmianach w grafiku.
- Wyświetlanie grafiku zajęć dla klientów i instruktorów.
- Zarządzanie danymi kursantów (dla administratora i instruktorów w ograniczonym zakresie).
- Informowanie o dostępności obiektu oraz ewentualnych ograniczeniach (np. remonty).
- Chat do komunikacji administratora i instruktora z klientem.

3.1 Ograniczenia projektu

W ramach projektu nie będą realizowane następujące funkcjonalności:

- System płatności online (np. integracja z operatorami płatności).
- Zaawansowane systemy rekomendacji lub analizy danych (np. AI do dopasowywania grup).
- Integracja z zewnętrznymi systemami (np. systemami basenu lub księgowości).

- Obsługa wielu oddziałów szkoły (system zakłada jedną lokalizację).

Zakres projektu został ograniczony w celu skupienia się na kluczowych funkcjonalnościach wspierających zarządzanie szkołą pływania oraz zapewnienia możliwości jego realizacji w założonym czasie.

4 Historyjki użytkownika

Na podstawie zebranego wywiadu z potencjalnymi klientami oraz zakresu usług realizowanych przez system sporządzono funkcje oraz grupy funkcji (historyjki użytkownika i epiki):

Historyjki użytkownika (Miro):

Historyjki użytkownika

5 Kryteria akceptacji

Historyjki użytkownika częściowo uzupełniono, tworząc następujące kryteria akceptacji:

Historyjka: Jako klient szkoły chcę zapisać się na zajęcia w dogodnym dla siebie terminie.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu.

Proces:

- Użytkownik wchodzi w zakładkę zapisu i widzi tylko dostępne terminy zajęć (wolne miejsca).
- Użytkownik wybiera termin klikając na niego i klika przycisk zapisz.
- System przypisuje użytkownika do konkretnej grupy zajęciowej.
- Po zapisie użytkownik otrzymuje potwierdzenie zapisu - powiadomienie oraz wiadomość mailowa o zapisie.
- Użytkownik może anulować zapis na zajęcia, po kliknięciu na zajęcia na które klient jest zapisany, pojawia się przycisk wypisz mnie.
- System blokuje możliwość zapisu na zajęcia, które są już pełne.

Kryteria нефunkcjonalne

- Czas odpowiedzi systemu na zapis - wysłania powiadomienia - nie powinien przekraczać 5 sekund. Czas wysłania wiadomości mailowej nie przekracza 5 minut.
- System działa prawidłowo w dużym obciążeniu, przy próbie 50 jednoczesnych zapisów.
- System prawidłowo zapisuje dane w bazie danych, baza danych obsługuje transakcje zabezpieczające mechanizm zapisywania się. Baza danych jest zabezpieczona przed wyciekami.

- Zapisy mają przyjazny interfejs użytkownika, wspierający osoby niepełnosprawne.

Scenariusze testowe

Scenariusz 1: Poprawny zapis na zajęcia

- **Given** użytkownik jest zalogowany i wybiera dostępne zajęcia.
- **When** użytkownik wybiera termin z wolnymi miejscami i klika „Zapisz się”
- **Then** system zapisuje użytkownika na zajęcia i wyświetla potwierdzenie

Scenariusz 2: Anulowanie zapisu

- **Given** użytkownik jest zapisany na zajęcia
- **When** użytkownik wybiera opcję „Anuluj zapis”
- **Then** system usuwa użytkownika z listy uczestników i potwierdza operację

Scenariusz 3: Równoczesny zapis wielu użytkowników

- **Given** dwóch użytkowników próbuje zapisać się na ostatnie wolne miejsce
- **When** obaj zatwierdzają zapis w tym samym czasie
- **Then** tylko jeden użytkownik zostaje zapisany, a drugi otrzymuje komunikat o braku miejsc

Historyjka: Jako uczestnik chcę mieć możliwość komunikacji z instruktorem i pracownikiem biura (obsługą).

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz jest zapisany na co najmniej jedno zajęcia.

Proces:

- Użytkownik wchodzi w zakładkę czatu.
- Użytkownik wybiera z listy kim chciałby chatować - może wybrać przypisanych do siebie instruktorów lub pracownika biura.
- Użytkownik wprowadza treść wiadomości i wysyła ją klikając przycisk "Wyślij" lub enter.
- System zapisuje wiadomość i zapisuje ją w bazie danych po czym dostarcza ją do adresata.
- Odbiorca (instruktor lub pracownik) może odpowiedzieć na wiadomość.
- Użytkownik otrzymuje powiadomienie o nowej wiadomości.
- System przechowuje historię wiadomości dostępną dla użytkownika.

Kryteria нефункциональные

- Czas dostarczenia wiadomości w systemie nie powinien przekraczać 5 sekund.
- System działa poprawnie przy jednoczesnej komunikacji wielu użytkowników (np. 50 aktywnych rozmów).
- Wiadomości są prawidłowo zapisywane w bazie danych i zabezpieczone przed wyciekami.
- Komunikacja jest zabezpieczona (szyfrowanie HTTPS, autoryzacja użytkowników).
- System zapewnia dostępność komunikacji na poziomie co najmniej 99%.
- Interfejs komunikacji jest intuicyjny, czytelny i dostosowany do urządzeń mobilnych oraz desktopowych.

Scenariusze testowe

Scenariusz 1: Wysłanie wiadomości do instruktora

- **Given** użytkownik jest zalogowany i ma przypisanego instruktora
- **When** użytkownik wpisuje wiadomość i klika „Wyślij”
- **Then** wiadomość zostaje zapisana i dostarczona do instruktora

Scenariusz 2: Odbiór wiadomości

- **Given** użytkownik otrzymał wiadomość
- **When** użytkownik wchodzi w zakładkę wiadomości
- **Then** system wyświetla nową wiadomość

Scenariusz 3: Odpowiedź na wiadomość

- **Given** użytkownik otrzymał wiadomość
- **When** odpowiada na wiadomość i klika „Wyślij”
- **Then** odpowiedź zostaje zapisana i dostarczona do nadawcy

Historyjka: Jako instruktor chcę mieć wgląd do grafiku, aby wiedzieć kiedy prowadzę lekcje.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę instruktora.

Proces:

- Użytkownik wchodzi w zakładkę grafiku.
- System wyświetla grafik zajęć przypisanych do instruktora.
- Grafik zawiera informacje o dacie, godzinie, grupie oraz przypisanym torze basenowym.
- Użytkownik może przeglądać grafik w podziale na dni, tygodnie lub miesiące.

- Użytkownik może kliknąć w konkretne zajęcia, aby zobaczyć szczegóły (lista uczestników, poziom grupy).
- System aktualizuje grafik w czasie rzeczywistym w przypadku zmian.

Kryteria niefunkcjonalne

- Czas ładowania grafiku nie powinien przekraczać 5 sekund.
- System działa poprawnie przy jednoczesnym dostępie wielu instruktorów (20 użytkowników).
- Dane grafiku są poprawnie pobierane i zapisywane w bazie danych oraz zabezpieczone przed utratą.
- System zapewnia aktualność danych (brak rozbieżności między bazą a widokiem użytkownika).
- Interfejs grafiku jest czytelny, intuicyjny i dostosowany do urządzeń mobilnych oraz desktopowych.

Scenariusze testowe

Scenariusz 1: Wyświetlenie grafiku instruktora

- **Given** użytkownik jest zalogowany jako instruktor
- **When** użytkownik wchodzi w zakładkę grafiku
- **Then** system wyświetla grafik zajęć przypisanych do instruktora

Scenariusz 2: Wyświetlenie szczegółów zajęć

- **Given** użytkownik przegląda grafik
- **When** użytkownik klika na konkretne zajęcia
- **Then** system wyświetla szczegóły zajęć

Scenariusz 3: Aktualizacja grafiku

- **Given** grafik instruktora jest wyświetlany
- **When** administrator zmienia termin zajęć
- **Then** system aktualizuje grafik i wyświetla nową informację

Historyjka: Jako pracownik biura chcę tworzyć harmonogram.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik wchodzi w zakładkę zarządzania harmonogramem.

- Użytkownik wybiera opcję dodania nowych zajęć.
- Użytkownik wprowadza dane zajęć: datę, godzinę, tor basenowy oraz przypisuje instruktora.
- System sprawdza dostępność wybranego toru oraz instruktora w danym terminie.
- W przypadku braku konfliktów system zapisuje zajęcia w harmonogramie.
- System przypisuje zajęcia do wybranego instruktora - instruktor otrzymuje powiadomienie.
- System aktualizuje grafik dostępny dla instruktorów i klientów.
- W przypadku konfliktu (np. zajęty tor lub instruktor) system wyświetla komunikat o błędzie i blokuje zapis.

Kryteria niefunkcjonalne

- Czas zapisu nowych zajęć nie powinien przekraczać 5 sekund.
- System działa poprawnie przy jednoczesnej edycji harmonogramu przez wielu administratorów (5 użytkowników).
- Dane harmonogramu są poprawnie zapisywane w bazie danych i zabezpieczone przed wyciekami.
- System zapewnia spójność danych (brak możliwości podwójnej rezerwacji toru lub instruktora).
- Interfejs zarządzania harmonogramem jest intuicyjny i czytelny..

Scenariusze testowe

Scenariusz 1: Dodanie nowych zajęć

- **Given** użytkownik jest zalogowany jako administrator
- **When** wprowadza dane zajęć i klika „Dodaj”
- **Then** system zapisuje zajęcia w harmonogramie

Scenariusz 2: Konflikt terminu instruktora

- **Given** instruktor ma już przypisane zajęcia w danym terminie
- **When** administrator próbuje przypisać go do nowych zajęć w tym samym czasie
- **Then** system blokuje zapis i wyświetla komunikat o konflikcie

Scenariusz 3: Konflikt toru basenowego

- **Given** tor basenowy jest zajęty w danym terminie
- **When** administrator próbuje dodać zajęcia na tym torze i godzinie
- **Then** system blokuje zapis i wyświetla komunikat o konflikcie

Scenariusz 4: Aktualizacja grafiku po dodaniu zajęć

- **Given** nowe zajęcia zostały dodane
- **When** instruktor lub klient otwiera grafik
- **Then** system wyświetla zaktualizowany harmonogram

Historyjka: Jako pracownik biura chcę edytować harmonogram oraz powiadamiać o tym instruktorów oraz klientów.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik wchodzi w zakładkę zarządzania harmonogramem.
- Użytkownik wybiera istniejące zajęcia do edycji.
- Użytkownik modyfikuje dane zajęć (np. godzinę, tor, instruktora).
- System sprawdza dostępność nowego terminu, toru oraz instruktora.
- W przypadku braku konfliktów system zapisuje zmiany w harmonogramie.
- System aktualizuje grafik dla instruktorów i klientów.
- System wysyła powiadomienia do przypisanych instruktorów oraz zapisanych klientów o zmianie w harmonogramie.
- W przypadku konfliktu system blokuje zapis zmian i wyświetla komunikat o błędzie.

Kryteria нефunkcjonalne

- Czas edycji i zapisania zmian nie powinien przekraczać 5 sekund.
- Powiadomienia w systemie są dostarczane w czasie do 5 sekund a wiadomości mailowe do 5 minut.
- System działa poprawnie przy jednoczesnej edycji harmonogramu przez wielu administratorów (5 użytkowników).
- Dane są poprawnie zapisywane w bazie danych i zabezpieczone przed utratą.
- System zapewnia spójność danych (brak konfliktów terminów, torów i instruktorów).
- Interfejs edycji harmonogramu jest intuicyjny i czytelny.

Scenariusze testowe

Scenariusz 1: Edycja zajęć

- **Given** administrator jest zalogowany i wybiera istniejące zajęcia
- **When** zmienia dane zajęć i klika „Zapisz”, edycja nie koliduje z niczym innym.

- **Then** system zapisuje zmiany w harmonogramie

Scenariusz 2: Powiadomienie użytkowników

- **Given** zajęcia zostały zmodyfikowane.
- **When** zmiany zostają zapisane.
- **Then** system wysyła powiadomienia do instruktorów i klientów

Scenariusz 3: Konflikt podczas edycji

- **Given** nowy termin koliduje z innymi zajęciami
- **When** administrator próbuje zapisać zmiany
- **Then** system blokuje zapis i wyświetla komunikat o konflikcie

Scenariusz 4: Aktualizacja grafiku

- **Given** zmiany w harmonogramie zostały zapisane
- **When** użytkownik otwiera grafik
- **Then** system wyświetla zaktualizowane dane

Historyjka: Jako pracownik biura chcę tworzyć nowe zajęcia (data, godzina, instruktor, tor), aby budować harmonogram.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik wchodzi w zakładkę zarządzania harmonogramem.
- Użytkownik wybiera opcję dodania nowych zajęć.
- Użytkownik wprowadza dane zajęć (data, godzina, instruktor, tor).
- System sprawdza dostępność wybranego instruktora oraz toru w podanym terminie.
- System weryfikuje poprawność wprowadzonych danych.
- W przypadku braku konfliktów system zapisuje nowe zajęcia w harmonogramie, i wyświetla komunikat o powodzeniu.
- System aktualizuje grafik instruktorów oraz dostępność torów.
- W przypadku konfliktu terminów lub błędnych danych system blokuje utworzenie zajęć i wyświetla komunikat o błędzie.

Kryteria нефункционалне

- Czas tworzenia i zapisania nowych zajęć nie powinien przekraczać 5 sekund.

- System umożliwia jednoczesne tworzenie zajęć przez wielu administratorów (5 użytkowników).
- Dane nowych zajęć są poprawnie zapisywane w bazie danych i zabezpieczone przed utratą.
- System zapewnia spójność danych (brak konfliktów terminów, torów i instruktorów).
- Interfejs dodawania zajęć jest intuicyjny i czytelny.
- System waliduje poprawność wprowadzanych danych przed zapisaniem zajęć.

Scenariusze testowe

Scenariusz 1: Utworzenie nowych zajęć

- **Given** administrator jest zalogowany i otwiera formularz dodawania zajęć
- **When** wprowadza poprawne dane zajęć i klika „Zapisz”
- **Then** system zapisuje nowe zajęcia w harmonogramie

Scenariusz 2: Aktualizacja harmonogramu

- **Given** nowe zajęcia zostały utworzone
- **When** użytkownik otwiera harmonogram
- **Then** system wyświetla nowe zajęcia w grafiku

Scenariusz 3: Konflikt instruktora lub toru

- **Given** wybrany instruktor lub tor jest już zajęty w podanym terminie
- **When** administrator próbuje utworzyć nowe zajęcia
- **Then** system blokuje zapis i wyświetla komunikat o konflikcie

Scenariusz 4: Walidacja danych formularza

- **Given** administrator wypełnia formularz dodawania zajęć
- **When** pozostawia wymagane pola puste lub wpisuje błędne dane
- **Then** system wyświetla komunikaty walidacyjne i nie zapisuje zajęć

Historyjka: Jako pracownik biura chcę usuwać zajęcia, aby reagować na sytuacje wyjątkowe.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik przechodzi do harmonogramu zajęć.

- Użytkownik wybiera zajęcia do usunięcia.
- System wyświetla okno potwierdzenia usunięcia.
- Użytkownik potwierdza usunięcie zajęć.
- System usuwa zajęcia z harmonogramu.
- System aktualizuje grafik instruktorów oraz dostępność torów.
- System wyświetla komunikat o poprawnym usunięciu zajęć.

Kryteria niefunkcjonalne

- Czas usunięcia zajęć nie powinien przekraczać 5 sekund.
- System zapewnia spójność danych po usunięciu zajęć.
- Dane są poprawnie aktualizowane w bazie danych.
- Interfejs usuwania zajęć jest prosty i czytelny.

Scenariusze testowe

Scenariusz 1: Usunięcie zajęć

- **Given** administrator wybiera istniejące zajęcia
- **When** potwierdza ich usunięcie
- **Then** system usuwa zajęcia z harmonogramu

Scenariusz 2: Aktualizacja harmonogramu

- **Given** zajęcia zostały usunięte
- **When** użytkownik otwiera harmonogram
- **Then** system nie wyświetla usuniętych zajęć

Scenariusz 3: Anulowanie usunięcia

- **Given** administrator wybiera opcję usunięcia zajęć
- **When** anuluje operację
- **Then** system pozostawia zajęcia bez zmian

Historyjka: Jako pracownik biura (administrator) chcę nadawać role (instruktor, pracownik biura) użytkownikom w celu kontroli dostępu.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora.

Proces:

- Administrator przechodzi do panelu zarządzania użytkownikami.
- Administrator wybiera użytkownika z listy.
- Administrator wybiera nową rolę dla użytkownika.
- System sprawdza poprawność uprawnień administratora.
- System zapisuje zmiany ról użytkownika.
- System aktualizuje poziom dostępu użytkownika.
- System wyświetla komunikat o poprawnym nadaniu roli.

Kryteria niefunkcjonalne

- Czas zmiany roli użytkownika nie powinien przekraczać 5 sekund.
- System zapewnia bezpieczeństwo i poprawność uprawnień użytkowników.
- Zmiany ról są poprawnie zapisywane w bazie danych.
- Interfejs zarządzania rolami jest prosty i czytelny.

Scenariusze testowe

Scenariusz 1: Nadanie roli użytkownikowi

- **Given** administrator wybiera użytkownika
- **When** nadaje mu nową rolę i zapisuje zmiany
- **Then** system aktualizuje uprawnienia użytkownika

Scenariusz 2: Brak uprawnień

- **Given** użytkownik bez roli administratora próbuje zmienić rolę
- **When** wykonuje operację nadania roli
- **Then** system blokuje dostęp i wyświetla komunikat o braku uprawnień

Scenariusz 3: Aktualizacja dostępu

- **Given** rola użytkownika została zmieniona
- **When** użytkownik loguje się ponownie
- **Then** system przydziela dostęp zgodny z nową rolą

Historyjka: Jako pracownik biura chcę wysyłać powiadomienia i komunikaty do klientów oraz instruktorów, w celu informowania o zmianach, remontach lub ograniczeniach.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik przechodzi do panelu komunikatów.
- Użytkownik tworzy nową wiadomość lub powiadomienie.
- Użytkownik wybiera grupę odbiorców (klienci, instruktorzy lub wszyscy).
- Użytkownik wpisuje treść komunikatu i zatwierdza wysyłkę.
- System zapisuje komunikat oraz wysyła powiadomienia do wybranych użytkowników.
- System wyświetla potwierdzenie poprawnego wysłania wiadomości.

Kryteria niefunkcjonalne

- Powiadomienia systemowe są dostarczane w czasie do 5 sekund.
- Wiadomości mailowe są wysyłane w czasie do 5 minut.
- System zapewnia poprawność dostarczenia komunikatów do wybranych odbiorców.
- Interfejs tworzenia komunikatów jest intuicyjny i czytelny.

Scenariusze testowe

Scenariusz 1: Wysłanie komunikatu

- **Given** administrator tworzy nowy komunikat
- **When** wybiera odbiorców i zatwierdza wysyłkę
- **Then** system wysyła powiadomienia do wybranych użytkowników

Scenariusz 2: Komunikat do wszystkich użytkowników

- **Given** administrator wybiera wszystkich odbiorców
- **When** wysyła komunikat
- **Then** system dostarcza wiadomość klientom oraz instruktorom

Scenariusz 3: Brak treści komunikatu

- **Given** administrator tworzy nowy komunikat
- **When** próbuje wysłać pustą wiadomość
- **Then** system wyświetla komunikat walidacyjny i blokuje wysyłkę

Historyjka: Jako pracownik biura chcę przydzielać instruktorów lub tory do zajęć, które mają je jeszcze nieprzydzielone.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do serwisu oraz posiada rolę administratora (pracownik biura).

Proces:

- Użytkownik przechodzi do harmonogramu zajęć.
- Użytkownik wybiera zajęcia bez przypisanego instruktora lub toru.
- Użytkownik wybiera dostępnego instruktora lub tor.
- System sprawdza dostępność wybranego instruktora oraz toru.
- W przypadku braku konfliktów system zapisuje przydział.
- System aktualizuje harmonogram zajęć.
- System wyświetla komunikat o poprawnym przypisaniu zasobów.

Kryteria niefunkcjonalne

- Czas przypisania instruktora lub toru nie powinien przekraczać 5 sekund.
- System zapewnia spójność danych i brak konfliktów terminów.
- Zmiany są poprawnie zapisywane w bazie danych.
- Interfejs przypisywania zasobów jest intuicyjny i czytelny.

Scenariusze testowe

Scenariusz 1: Przypisanie instruktora

- **Given** istnieją zajęcia bez przypisanego instruktora
- **When** administrator wybiera instruktora i zapisuje zmiany
- **Then** system przypisuje instruktora do zajęć

Scenariusz 2: Przypisanie toru

- **Given** istnieją zajęcia bez przypisanego toru
- **When** administrator wybiera tor i zapisuje zmiany
- **Then** system przypisuje tor do zajęć

Scenariusz 3: Konflikt dostępności

- **Given** wybrany instruktor lub tor jest zajęty
- **When** administrator próbuje zapisać przydział
- **Then** system blokuje zapis i wyświetla komunikat o konflikcie

Historyjka: Jako klient chcę zalogować się do systemu, aby uzyskać dostęp do swojego profilu i zapisów w harmonogramie.

Kryteria funkcjonalne

Warunki wstępne: Klient posiada aktywne konto w systemie.

Proces:

- Użytkownik przechodzi do formularza logowania.
- Użytkownik wprowadza login oraz hasło.
- System weryfikuje poprawność danych logowania.
- W przypadku poprawnych danych system loguje użytkownika.
- System przekierowuje użytkownika do panelu klienta.
- W przypadku błędnych danych system wyświetla komunikat o błędzie.

Kryteria нефункционалне

- Czas logowania nie powinien przekraczać 5 sekund.
- Dane logowania są przesyłane i przechowywane w sposób bezpieczny.
- System zapewnia ochronę przed nieautoryzowanym dostępem.
- Interfejs logowania jest prosty i czytelny.

Scenariusze testowe

Scenariusz 1: Poprawne logowanie

- **Given** klient posiada aktywne konto
- **When** wprowadza poprawny login i hasło
- **Then** system loguje użytkownika i wyświetla panel klienta

Scenariusz 2: Błędne dane logowania

- **Given** klient znajduje się na stronie logowania
- **When** wprowadza niepoprawny login lub hasło
- **Then** system wyświetla komunikat o błędzie i nie loguje użytkownika

Scenariusz 3: Dostęp do harmonogramu

- **Given** klient został poprawnie zalogowany
- **When** przechodzi do harmonogramu
- **Then** system wyświetla jego zapisy na zajęcia

Historyjka: Jako klient chcę przeglądać listę dostępnych zajęć z podziałem na dni i godziny, aby wybrać odpowiedni termin dla siebie.

Kryteria funkcjonalne

Warunki wstępne: Klient jest zalogowany do systemu.

Proces:

- Użytkownik przechodzi do harmonogramu dostępnych zajęć.

- System pobiera aktualną listę zajęć.
- System wyświetla zajęcia z podziałem na dni oraz godziny.
- Użytkownik może przeglądać szczegóły zajęć (instruktor, tor, liczba miejsc).
- Użytkownik wybiera interesujący go termin zajęć.

Kryteria niefunkcjonalne

- Czas wyświetlenia harmonogramu nie powinien przekraczać 5 sekund.
- System wyświetla wyłącznie aktualne i dostępne zajęcia.
- Dane harmonogramu są zgodne z aktualnym planem zajęć.
- Interfejs harmonogramu jest czytelny i intuicyjny.

Scenariusze testowe

Scenariusz 1: Wyświetlenie listy zajęć

- **Given** klient jest zalogowany do systemu
- **When** otwiera harmonogram zajęć
- **Then** system wyświetla listę dostępnych zajęć z podziałem na dni i godziny

Scenariusz 2: Wyświetlenie szczegółów zajęć

- **Given** klient przegląda harmonogram zajęć
- **When** wybiera konkretne zajęcia
- **Then** system wyświetla szczegóły wybranego terminu

Scenariusz 3: Brak dostępnych zajęć

- **Given** w systemie nie ma dostępnych zajęć
- **When** klient otwiera harmonogram
- **Then** system wyświetla informację o braku dostępnych terminów

Historyjka: Jako klient chcę móc anulować zapis na zajęcia, aby móc zwolnić miejsce.

Kryteria funkcjonalne

Warunki wstępne: Klient jest zalogowany do systemu oraz zapisany na zajęcia.

Proces:

- Użytkownik przechodzi do swojego harmonogramu zajęć.
- Użytkownik wybiera zajęcia, z których chce się wypisać.
- System wyświetla opcję anulowania zapisu.

- Użytkownik potwierdza anulowanie zapisu.
- System usuwa klienta z listy uczestników zajęć.
- System aktualizuje liczbę dostępnych miejsc.
- System wyświetla komunikat o poprawnym anulowaniu zapisu.

Kryteria niefunkcjonalne

- Czas anulowania zapisu nie powinien przekraczać 5 sekund.
- System poprawnie aktualizuje liczbę wolnych miejsc.
- Dane są poprawnie zapisywane w bazie danych.
- Interfejs anulowania zapisu jest prosty i czytelny.

Scenariusze testowe

Scenariusz 1: Anulowanie zapisu

- **Given** klient jest zapisany na zajęcia
- **When** wybiera opcję anulowania zapisu i potwierdza operację
- **Then** system usuwa klienta z listy uczestników

Scenariusz 2: Aktualizacja liczby miejsc

- **Given** klient anulował zapis na zajęcia
- **When** inny użytkownik przegląda dostępne zajęcia
- **Then** system wyświetla zaktualizowaną liczbę wolnych miejsc

Scenariusz 3: Anulowanie operacji

- **Given** klient wybiera opcję wypisania się z zajęć
- **When** anuluje operację
- **Then** system pozostawia zapis na zajęcia bez zmian

Historyjka: Jako klient chcę widzieć szczegóły zajęć, na które jestem zapisany (instruktor, tor, godzina), aby być przygotowanym.

Kryteria funkcjonalne

Warunki wstępne: Klient jest zalogowany do systemu oraz zapisany na co najmniej jedno zajęcia.

Proces:

- Użytkownik przechodzi do swojego harmonogramu.
- System pobiera listę zajęć przypisanych do klienta.

- System wyświetla listę zapisanych zajęć.
- Użytkownik wybiera konkretne zajęcia.
- System wyświetla szczegóły zajęć (instruktor, tor, data, godzina).

Kryteria нефункционалне

- Czas wyświetlenia szczegółów zajęć nie powinien przekraczać 5 sekund.
- System wyświetla aktualne dane zgodne z harmonogramem.
- Dostęp do szczegółów jest ograniczony do zalogowanego użytkownika.
- Interfejs prezentacji szczegółów zajęć jest czytelny i intuicyjny.

Scenariusze testowe

Scenariusz 1: Wyświetlenie szczegółów zajęć

- **Given** klient jest zapisany na zajęcia
- **When** wybiera konkretne zajęcia w harmonogramie
- **Then** system wyświetla szczegóły (instruktor, tor, godzina)

Scenariusz 2: Brak zapisów na zajęcia

- **Given** klient nie jest zapisany na żadne zajęcia
- **When** otwiera harmonogram
- **Then** system informuje o braku zapisów

Scenariusz 3: Aktualizacja danych zajęć

- **Given** dane zajęć zostały zmienione przez administratora
- **When** klient otwiera szczegóły zajęć
- **Then** system wyświetla zaktualizowane informacje

Historyjka: Jako klient chcę odzyskać hasło, aby móc ponownie uzyskać dostęp do konta.

Kryteria funkcjonalne

Warunki wstępne: Klient posiada konto w systemie i ma dostęp do adresu e-mail przypisanego do konta.

Proces:

- Użytkownik przechodzi do formularza odzyskiwania hasła.
- Użytkownik podaje adres e-mail powiązany z kontem.
- System weryfikuje poprawność adresu e-mail.
- System generuje link do resetu hasła i wysyła go na e-mail użytkownika.

- Użytkownik otwiera link i ustawia nowe hasło.
- System zapisuje nowe hasło i umożliwia logowanie.
- System wyświetla komunikat o poprawnej zmianie hasła.

Kryteria niefunkcjonalne

- Link do resetu hasła powinien być ważny przez określony czas (np. 15 minut).
- Czas wysłania e-maila nie powinien przekraczać 5 minut.
- System zapewnia bezpieczeństwo procesu odzyskiwania hasła.
- Interfejs odzyskiwania hasła jest prosty i czytelny.

Scenariusze testowe

Scenariusz 1: Poprawne odzyskanie hasła

- **Given** klient wprowadza poprawny adres e-mail
- **When** wysłał żądanie resetu hasła
- **Then** system wysłał link do resetu hasła na e-mail

Scenariusz 2: Nieistniejący e-mail

- **Given** użytkownik podaje adres e-mail
- **When** adres nie istnieje w systemie
- **Then** system wyświetla komunikat o braku konta

Scenariusz 3: Zmiana hasła

- **Given** użytkownik otrzymał aktywny link do resetu hasła
- **When** ustawia nowe hasło i zatwierdza
- **Then** system zapisuje nowe hasło i umożliwia logowanie

Historyjka: Jako klient chcę otrzymywać wszystkie istotne komunikaty i powiadomienia o zmianach w harmonogramie oraz ograniczeniach obiektu.

Kryteria funkcjonalne

Warunki wstępne: Klient posiada aktywne konto w systemie oraz ma włączone powiadomienia.

Proces:

- Administrator tworzy komunikat lub powiadomienie.
- System identyfikuje użytkowników, których dotyczą zmiany.
- System wysłał powiadomienie do klientów.

- Użytkownik otrzymuje powiadomienie w systemie oraz/lub e-mail.
- Użytkownik może wyświetlić szczegóły komunikatu.

Kryteria niefunkcjonalne

- Powiadomienia systemowe są dostarczane w czasie do 5 sekund.
- Wiadomości e-mail są dostarczane w czasie do 5 minut.
- System zapewnia dostarczenie komunikatów do właściwych odbiorców.
- Użytkownik ma możliwość łatwego przeglądania historii powiadomień.

Scenariusze testowe

Scenariusz 1: Otrzymanie powiadomienia o zmianie

- **Given** administrator wysłał komunikat o zmianie w harmonogramie
- **When** system przetwarza komunikat
- **Then** klient otrzymuje powiadomienie

Scenariusz 2: Otrzymanie komunikatu o ograniczeniach obiektu

- **Given** administrator publikuje informację o ograniczeniach obiektu
- **When** komunikat jest wysyłany
- **Then** system dostarcza powiadomienie do klientów

Scenariusz 3: Przegląd historii powiadomień

- **Given** klient otrzymał wcześniejsze powiadomienia
- **When** otwiera sekcję powiadomień
- **Then** system wyświetla listę wszystkich istotnych komunikatów

Historyjka: Jako instruktor chcę widzieć harmonogram moich zajęć, aby wiedzieć kiedy prowadzę zajęcia.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do systemu oraz posiada rolę instruktora.

Proces:

- Instruktor przechodzi do swojego harmonogramu.
- System pobiera zajęcia przypisane do instruktora.
- System wyświetla listę zajęć z podziałem na dni i godziny.
- Instruktor może podejrzeć szczegóły zajęć (tor, liczba uczestników).

Kryteria niefunkcjonalne

- Czas wyświetlenia harmonogramu nie powinien przekraczać 5 sekund.
- System wyświetla wyłącznie zajęcia przypisane do zalogowanego instruktora.
- Dane harmonogramu są zgodne z aktualnym planem zajęć.
- Interfejs harmonogramu jest czytelny i intuicyjny.

Scenariusze testowe

Scenariusz 1: Wyświetlenie harmonogramu instruktora

- **Given** instruktor jest zalogowany do systemu
- **When** przechodzi do swojego harmonogramu
- **Then** system wyświetla jego zaplanowane zajęcia

Scenariusz 2: Brak przypisanych zajęć

- **Given** instruktor nie ma przypisanych zajęć
- **When** otwiera harmonogram
- **Then** system wyświetla informację o braku zajęć

Scenariusz 3: Aktualizacja harmonogramu

- **Given** harmonogram instruktora został zmieniony
- **When** instruktor ponownie otwiera harmonogram
- **Then** system wyświetla zaktualizowane dane

Historyjka: Jako instruktor chcę widzieć listę kursantów zapisanych na dane zajęcia i móc po kliknięciu zobaczyć informacje o kliencie, takie jak poziom zaawansowania, aby dostosować trening.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do systemu oraz posiada rolę instruktora i ma przypisane zajęcia.

Proces:

- Instruktor przechodzi do swojego harmonogramu.
- Instruktor wybiera konkretne zajęcia.
- System wyświetla listę zapisanych kursantów.
- Instruktor wybiera konkretnego klienta z listy.
- System wyświetla szczegóły klienta (np. poziom zaawansowania, podstawowe informacje).

Kryteria нефункционалне

- Czas wyświetlenia listy kursantów i danych klienta nie powinien przekraczać 2 sekund.
- Dostęp do danych klientów jest ograniczony wyłącznie do przypisanych instruktorów.
- Dane są aktualne i zgodne z systemem zapisów.
- Interfejs prezentacji listy kursantów i szczegółów klienta jest czytelny i intuicyjny.

Scenariusze testowe

Scenariusz 1: Wyświetlenie listy kursantów

- **Given** instruktor wybiera zajęcia
- **When** otwiera listę uczestników
- **Then** system wyświetla kursantów zapisanych na zajęcia

Scenariusz 2: Wyświetlenie szczegółów klienta

- **Given** instruktor widzi listę kursantów
- **When** wybiera konkretnego klienta
- **Then** system wyświetla informacje o kliencie, w tym poziom zaawansowania

Scenariusz 3: Brak kursantów na zajęciach

- **Given** na dane zajęcia nie zapisano żadnych klientów
- **When** instruktor otwiera listę uczestników
- **Then** system wyświetla informację o braku kursantów

Historyjka: Jako instruktor chcę być powiadamiany o zmianach w swoim grafiku, a także o nowych zajęciach i nowych kursantach, aby na bieżąco reagować na aktualizacje i odpowiednio przygotować się do prowadzenia zajęć.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do systemu oraz posiada rolę instruktora i ma przypisane zajęcia.

Proces:

- System monitoruje zmiany w harmonogramie zajęć przypisanych do instruktora.
- System wykrywa modyfikacje takie jak zmiana godziny, toru, daty lub listy uczestników.
- System identyfikuje instruktora jako odbiorcę powiadomienia.
- System generuje odpowiedni komunikat (zmiana grafiku, nowe zajęcia, nowi kursanci).

- System wysyła powiadomienie w aplikacji oraz opcjonalnie e-mail.
- Instruktor może otworzyć powiadomienie i przejść do szczegółów zajęć.
- System archiwizuje wysłane powiadomienia w historii użytkownika.

Kryteria niefunkcjonalne

- Powiadomienia systemowe są dostarczane w czasie do 5 sekund od wystąpienia zdarzenia.
- Wiadomości e-mail są wysyłane w czasie do 5 minut.
- System zapewnia niezawodne dostarczenie powiadomień do właściwego instruktora.
- Powiadomienia są dostępne w historii przez minimum 30 dni.
- Interfejs powiadomień jest czytelny i umożliwia szybkie przejście do szczegółów zajęć.

Scenariusze testowe

Scenariusz 1: Powiadomienie o zmianie grafiku

- **Given** harmonogram instruktora został zmodyfikowany przez administratora
- **When** system zapisuje zmiany w zajęciach
- **Then** instruktor otrzymuje powiadomienie o zmianie terminu lub szczegółów zajęć

Scenariusz 2: Powiadomienie o nowych zajęciach

- **Given** administrator przypisuje nowe zajęcia do instruktora
- **When** zajęcia zostają zapisane w systemie
- **Then** instruktor otrzymuje informację o nowych zajęciach wraz z ich szczegółami

Scenariusz 3: Powiadomienie o nowych kursantach

- **Given** do zajęć zostali zapisani nowi kursanci
- **When** system aktualizuje listę uczestników
- **Then** instruktor otrzymuje powiadomienie o nowych zapisach wraz z liczbą uczestników

Historyjka: Jako instruktor chcę móc komunikować się ze swoimi klientami przy użyciu czatu, aby móc odpowiedzieć na ich pytania.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do systemu oraz posiada rolę instruktora, a także ma przypisanych klientów do zajęć.

Proces:

- Instruktor przechodzi do sekcji czatu.
- System wyświetla listę dostępnych rozmów z klientami.
- Instruktor wybiera konkretnego klienta.
- System otwiera okno konwersacji.
- Instruktor wysyła wiadomość do klienta.
- System zapisuje wiadomość i dostarcza ją do klienta.
- Klient może odpowiedzieć w ramach tej samej konwersacji.
- System wyświetla historię rozmowy w chronologicznej kolejności.

Kryteria niefunkcjonalne

- Wiadomości są dostarczane w czasie rzeczywistym (do kilku sekund).
- System zapewnia poufność i bezpieczeństwo komunikacji.
- Historia czatu jest dostępna dla obu stron rozmowy.
- Interfejs czatu jest czytelny i umożliwia łatwą wymianę wiadomości.

Scenariusze testowe

Scenariusz 1: Wysłanie wiadomości do klienta

- **Given** instruktor otwiera czat z klientem
- **When** wysyła wiadomość
- **Then** klient otrzymuje wiadomość w czasie rzeczywistym

Scenariusz 2: Odbiór odpowiedzi od klienta

- **Given** klient odpowiada w czacie
- **When** wiadomość zostaje wysłana
- **Then** instruktor widzi odpowiedź w konwersacji

Scenariusz 3: Wyświetlenie historii rozmowy

- **Given** istnieje historia czatu
- **When** instruktor otwiera konwersację
- **Then** system wyświetla pełną historię wiadomości w kolejności chronologicznej

Historyjka: Jako pracownik biura chcę otrzymywać powiadomienia o pytaniach klientów, aby móc szybko odpowiadać na ich zapytania.

Kryteria funkcjonalne

Warunki wstępne: Użytkownik jest zalogowany do systemu oraz posiada rolę pracownika biura (administratora).

Proces:

- Klient wysyła pytanie poprzez system (np. czat lub formularz kontaktowy).
- System rejestruje nowe zgłoszenie od klienta.
- System identyfikuje pracownika biura jako odbiorcę powiadomienia.
- System generuje powiadomienie o nowym pytaniu.
- System wysyła powiadomienie w aplikacji oraz/lub e-mail.
- Pracownik biura otrzymuje powiadomienie i może przejść do czatu z klientem.
- System otwiera okno konwersacji z danym klientem.
- Pracownik może odpowiedzieć na pytanie w ramach czatu.

Kryteria нефunkcjonalne

- Powiadomienia systemowe są dostarczane w czasie do 5 sekund od wysłania pytania przez klienta.
- Wiadomości e-mail są dostarczane w czasie do 5 minut.
- System zapewnia przypisanie zgłoszeń do właściwego pracownika biura.
- Historia zgłoszeń i czatu jest dostępna w systemie.
- Interfejs czatu umożliwia szybkie przejście z powiadomienia do rozmowy.

Scenariusze testowe

Scenariusz 1: Otrzymanie powiadomienia o pytaniu klienta

- **Given** klient wysyła pytanie przez system
- **When** system rejestruje zgłoszenie
- **Then** pracownik biura otrzymuje powiadomienie

Scenariusz 2: Przejście do czatu z klientem

- **Given** pracownik otrzymał powiadomienie o pytaniu
- **When** klika w powiadomienie
- **Then** system otwiera czat z odpowiednim klientem

Scenariusz 3: Odpowiedź na pytanie

- **Given** otwarty jest czat z klientem
- **When** pracownik wysyła odpowiedź
- **Then** klient otrzymuje wiadomość w konwersacji

6 Architektura logiczna projektu

Cały ten punkt dokumentacji stanowi opis jak projekt zostałby zaimplementowany w całości w wersji produkcyjnej. W praktyce na obecnym etapie implementacja stanowi uproszczoną wersję - prototyp interfejsu użytkownika oraz aplikacja realizująca część z funkcjonalności backendowych w uproszczony sposób - bez bazy danych, w pamięci, w czystej Javie.

Projekt *System Zarządzania Szkołą Pływania „Delfinek”* oparty jest na trójwarstwowej architekturze klient-serwer. Centralnym elementem systemu w pełnej wersji implementacyjnej byłby serwer aplikacyjny zrealizowany w technologii **Java** z frameworkiem **Spring Boot**, komunikujący się z relacyjną bazą danych **PostgreSQL** za pośrednictwem warstwy persystencji **JPA/Hibernate**. Warstwa kliencka obejmuje aplikację webową dostępną z poziomu przeglądarki oraz dedykowany panel administratora zintegrowany w tej samej aplikacji. W praktyce zrealizowana została uproszczona wersja aplikacji serwerowej, dla prostoty napisana w czystej Javie 21, bez bazy danych, "baza danych" w pamięci aplikacji.

6.1 Warstwy systemu

System podzielony jest na trzy logiczne warstwy, zgodnie z klasycznym wzorcem architektury wielowarstwowej:

1. **Warstwa prezentacji (frontend)** – interfejs użytkownika dostępny w przeglądarce internetowej, skierowany do wszystkich ról: klientów szkoły, instruktorów oraz administratora (pracownika biura). Komunikuje się z backendem wyłącznie poprzez REST API.
2. **Warstwa logiki biznesowej (backend)** – serwer aplikacyjny Spring Boot realizujący całą logikę domenową systemu, udostępniający REST API w formacie JSON, obsługujący uwierzytelnianie i autoryzację opartą na rolach (RBAC).
3. **Warstwa danych (baza danych)** – relacyjna baza danych PostgreSQL przechowująca wszystkie dane systemu; dostęp realizowany przez JPA/Hibernate.

6.2 Warstwa prezentacji (frontend)

Aplikacja webowa stanowi jedyny interfejs użytkownika w bieżącym zakresie projektu. Dostępna jest z poziomu dowolnej przeglądarki internetowej bez konieczności instalacji.

- **Technologia:** HTML5, CSS3, JavaScript z frameworkiem (np. React lub Vue.js).
- **Komunikacja z backendem:** żądania HTTP(S) do REST API, wymiana danych w formacie JSON.
- **Widoki dostosowane do roli:** po zalogowaniu aplikacja prezentuje interfejs odpowiedni dla danej roli – klienta, instruktora lub administratora.
- **Funkcjonalności widoku klienta:** przeglądanie harmonogramu dostępnych zajęć, zapisywanie się i wypisywanie z zajęć, podgląd własnego grafiku, przeglądanie powiadomień, czat z instruktorem lub administratorem.

- **Funkcjonalności widoku instruktora:** podgląd własnego harmonogramu, lista kursantów przypisanych do zajęć, podgląd danych kursanta (poziom zaawansowania), czat z klientami.
- **Funkcjonalności widoku administratora:** zarządzanie harmonogramem (tworzenie, edycja, usuwanie zajęć), przydzielanie instruktorów i torów, zarządzanie rolami użytkowników, wysyłanie komunikatów i powiadomień, zarządzanie remontami torów.

6.3 Warstwa logiki biznesowej (backend)

Backend realizowany jest jako aplikacja **Spring Boot** (Java), działająca jako samodzielny serwer HTTP(S). Architektura wewnętrzna backendu opiera się na wzorcu warstwowym: kontrolery REST → serwisy → repozytoria.

- **Technologia:** Java, Spring Boot, Spring Web MVC, Spring Security.
- **REST API:** udostępnienie endpointów HTTP(S) z odpowiedziami w formacie JSON; wersjonowanie API (/api/v1/...).
- **Uwierzytelnianie i autoryzacja:** mechanizm JWT (JSON Web Token) – po zalogowaniu użytkownik otrzymuje token, który dołączany jest do każdego kolejnego żądania. Spring Security weryfikuje token i kontroluje dostęp na podstawie roli (RBAC): KLIENT, INSTRUKTOR, ADMINISTRATOR.
- **Logika domenowa:** serwisy realizują kluczowe operacje systemu – HarmonogramSerwis, ZapisSerwis, UzytkownikSerwis, PowiadomienieSerwis, ChatSerwis, TorSerwis, MailSerwis, TokenSerwis.
- **Walidacja danych:** po stronie backendu z użyciem Bean Validation (adnotacje @Valid, @NotNull itd.).
- **Powiadomienia e-mail:** wysyłka przez JavaMailSender (Spring Mail) – potwierdzenia zapisów, alerty o zmianach harmonogramu, linki do resetu hasła.
- **Obsługa błędów:** globalny handler wyjątków (@ControllerAdvice) zwracający ujednolicone odpowiedzi błędów w formacie JSON.

6.4 Warstwa danych (baza danych)

- **Silnik: PostgreSQL** – relacyjna baza danych obsługująca transakcje ACID, konieczne do zapewnienia spójności przy jednoczesnych operacjach (np. rywalizujące zapisy na ostatnie wolne miejsce w grupie).
- **Dostęp z backendu: JPA/Hibernate** – mapowanie obiektowo-relacyjne (ORM); repozytoria Spring Data JPA (JpaRepository) jako warstwa dostępu do danych.
- **Migracje schematu:** zarządzane narzędziem Flyway lub Liquibase, zapewniającym kontrolę wersji schematu bazy.
- **Zakres przechowywanych danych:**

- profile użytkowników (klienci, instruktorzy, administratorzy) wraz z hasłami w postaci skrótu (BCrypt),
 - definicje zajęć z przypisaniem instruktora, toru, typu i cykliczności,
 - harmonogramy zajęć i zapisy kursantów,
 - historia wiadomości czatu i konwersacji,
 - powiadomienia i ich status (przeczytane/nieprzeczytane),
 - dane torów basenowych wraz ze statusem (wolny, zajęty, remont).
- **Integralność danych:** klucze obce, ograniczenia unikalności, transakcje – zabezpieczenie przed podwójną rezerwacją toru lub instruktora w tym samym terminie.

6.5 Bezpieczeństwo

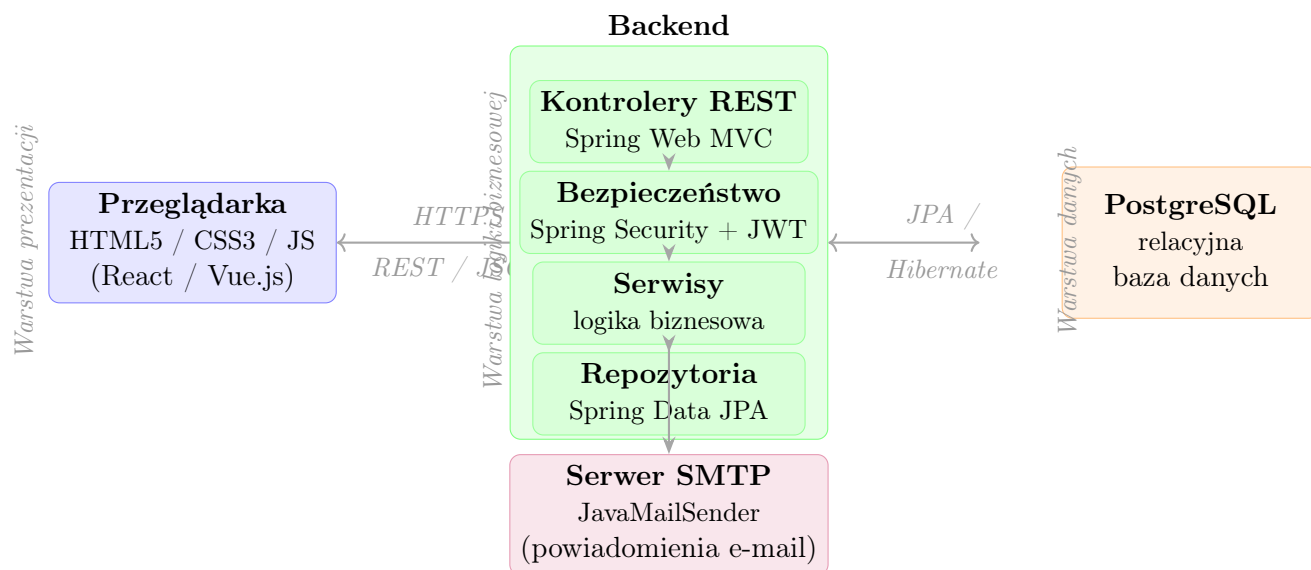
- **Transport:** komunikacja wyłącznie przez HTTPS (TLS).
- **Uwierzytelnianie:** JWT z określonym czasem ważności tokenu; mechanizm odświeżania tokenu (refresh token).
- **Autoryzacja:** Spring Security + RBAC – każdy endpoint chroniony odpowiednią rolą; użytkownik nie uzyska dostępu do zasobów innej roli.
- **Hasła:** przechowywane wyłącznie jako skróty BCrypt; proces resetowania hasła oparty na jednorazowym tokenie z czasem ważności 15 minut, wysyłanym na adres e-mail.
- **Walidacja wejścia:** dane wejściowe walidowane po stronie backendu niezależnie od frontendu.

6.6 Aplikacja mobilna

Aplikacja mobilna nie jest objęta bieżącym zakresem projektu. Ze względu na architekturę opartą na REST API, wdrożenie dedykowanej aplikacji mobilnej (np. w technologii Flutter lub React Native) jest możliwe w przyszłości bez zmian po stronie backendu – wystarczy podłączenie nowego klienta do istniejącego API.

6.7 Schemat architektury

Poniższy schemat przedstawia warstwową architekturę systemu oraz przepływ komunikacji między komponentami.



Rysunek 1: Schemat architektury systemu Delfinek

Wszystkie żądania użytkowników trafiają do kontrolerów REST, gdzie Spring Security weryfikuje token JWT i rolę użytkownika. Po pozytywnej autoryzacji żądanie przekazywane jest do odpowiedniego serwisu, który realizuje logikę biznesową i za pośrednictwem repozytoriów JPA odczytuje lub zapisuje dane w PostgreSQL. Odpowiedź wraca tą samą ścieżką w formacie JSON. Zdarzenia wymagające powiadomień e-mail (potwierdzenia zapisów, alerty o zmianach, reset hasła) obsługiwane są asynchronicznie przez **MailSerwis** z użyciem **JavaMailSender**.

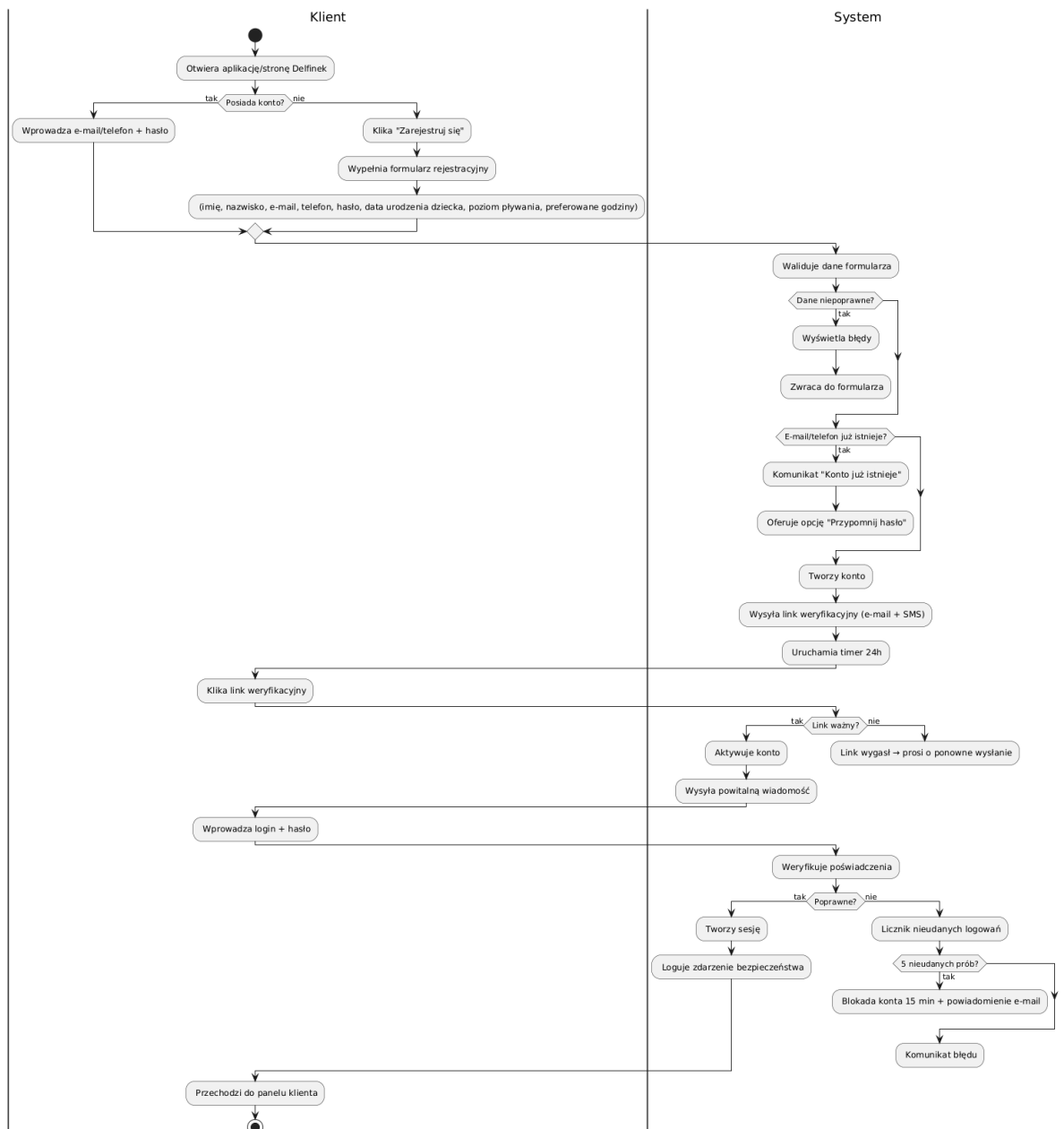
6.8 Stos technologiczny – podsumowanie

Obszar	Technologia	Rola w systemie
Język backendowy	Java (JDK 21)	Implementacja logiki biznesowej
Framework backend	Spring Boot	Serwer HTTP, IoC, konfiguracja
REST API	Spring Web MVC	Endpointy HTTP, serializacja JSON
Bezpieczeństwo	Spring Security + JWT	Uwierzytelnianie, autoryzacja RBAC
ORM	JPA / Hibernate	Mapowanie obiektowo-relacyjne
Baza danych	PostgreSQL	Persystencja danych, transakcje ACID
Migracje schematu	Flyway / Liquibase	Wersjonowanie schematu bazy
E-mail	Spring Mail (SMTP)	Powiadomienia, reset hasła
Frontend	HTML5/CSS3/JS	Interfejs użytkownika w przeglądarce

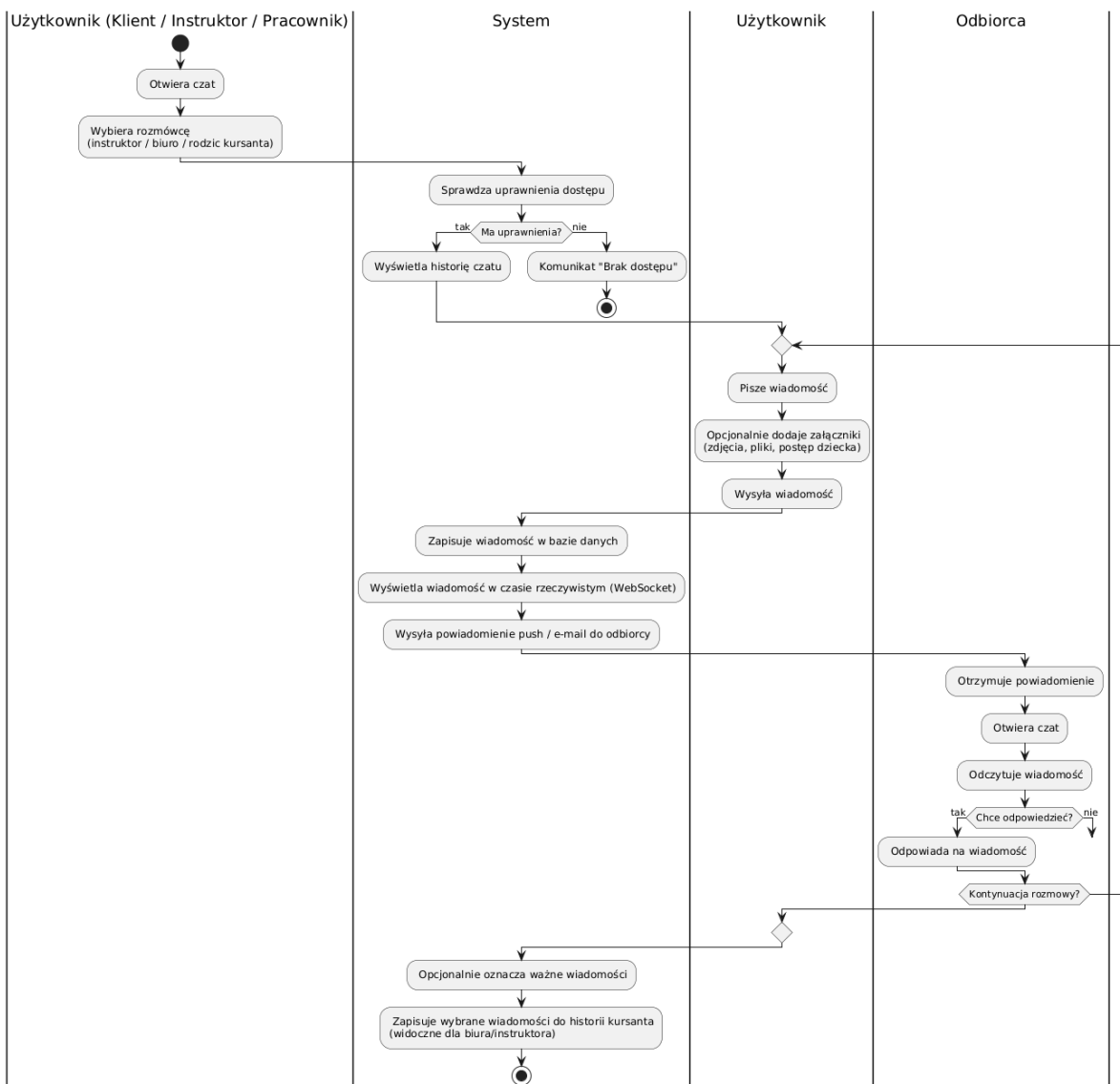
Tabela 1: Stos technologiczny systemu Delfinek

7 Diagramy aktywności

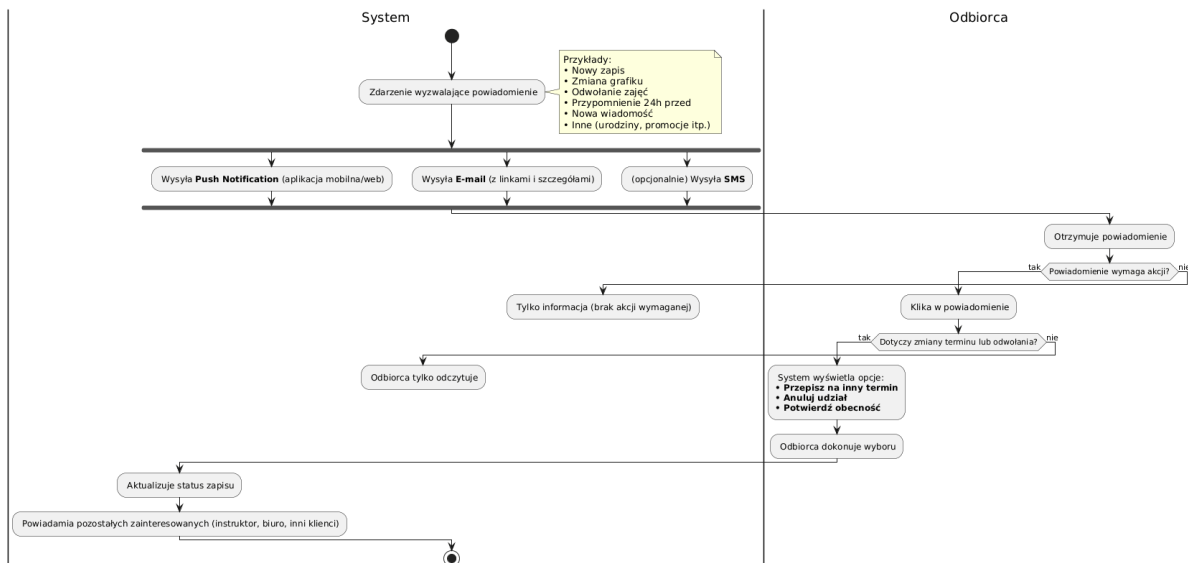
Poniżej przedstawiono pełen zestaw diagramów aktywności opisujących kluczowe procesy w systemie Delfinek: logowanie, komunikację (czat), powiadomienia, przeglądanie harmonogramu oraz zapisy na zajęcia. W repozytorium na githubie, do którego link podano na pierwszej stronie dokumentu, można pobrać załączone pliki png przedstawiające diagramy aktywności.



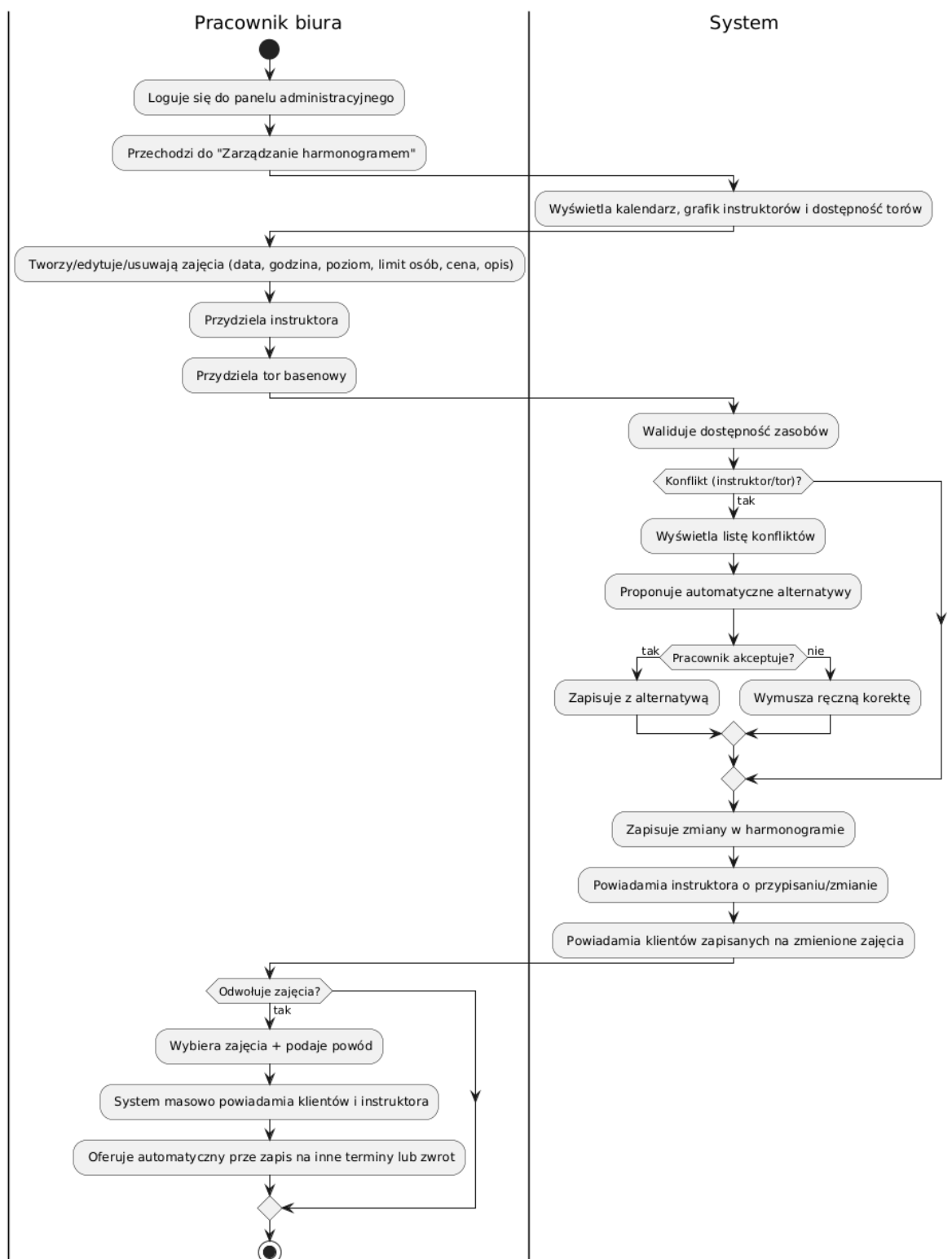
Rysunek 2: Logowanie i odzyskiwanie hasła



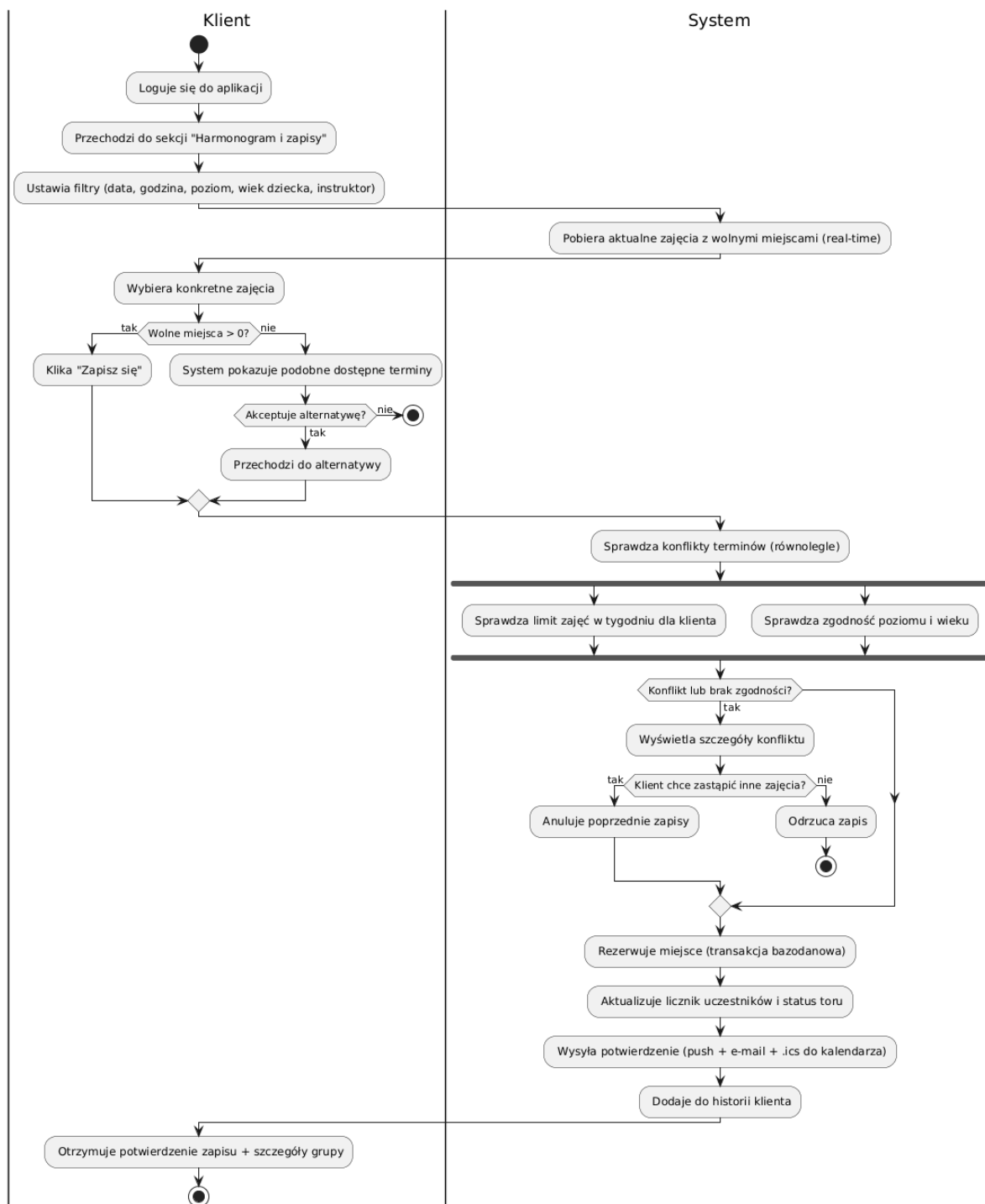
Rysunek 3: Czat



Rysunek 4: Powiadomienia



Rysunek 5: Przeglądanie harmonogramu



Rysunek 6: Zapis na zajęcia

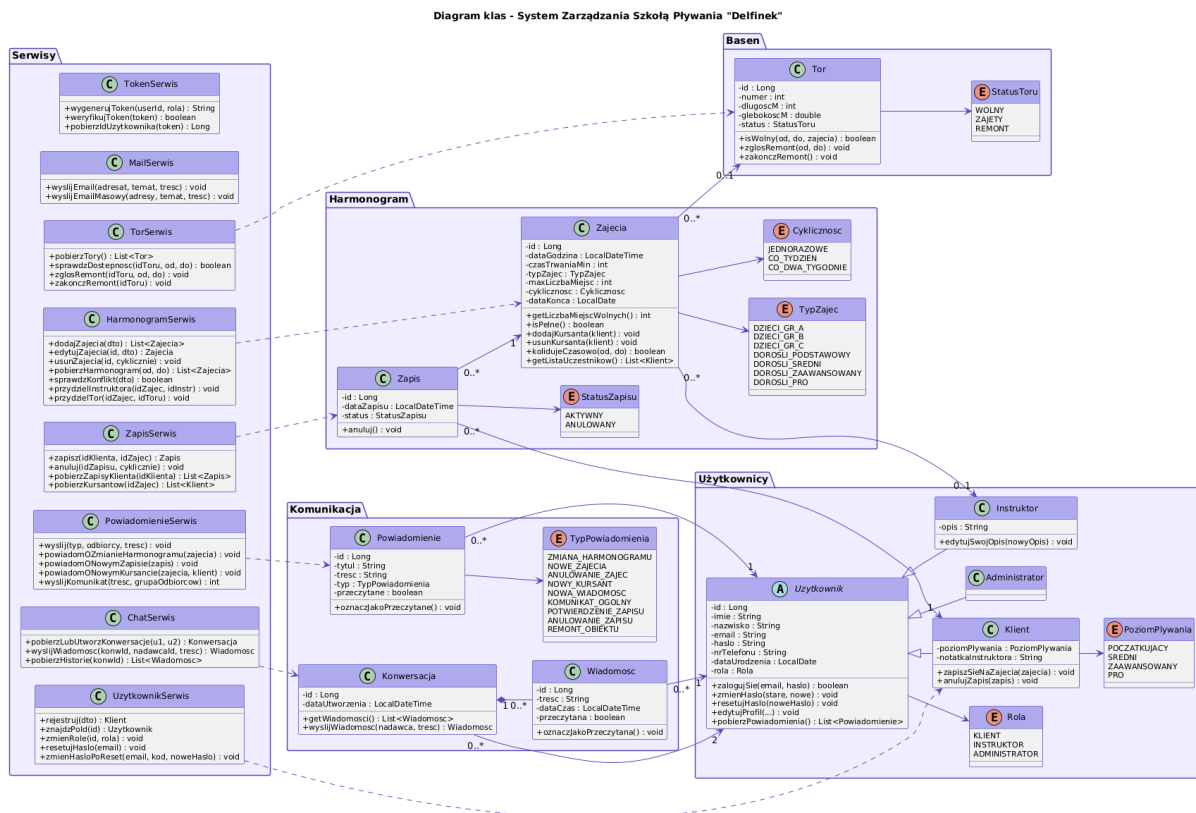
8 Prototyp UI

Utworzono lo-fi prototyp interfejsu użytkownika w programie Moqups. Link do pobrania pliku pdf tego interfejsu, można znaleźć na githubie, do którego link umieszczono na pierwszej stronie dokumentu.

9 Diagramy klas i sekwencji

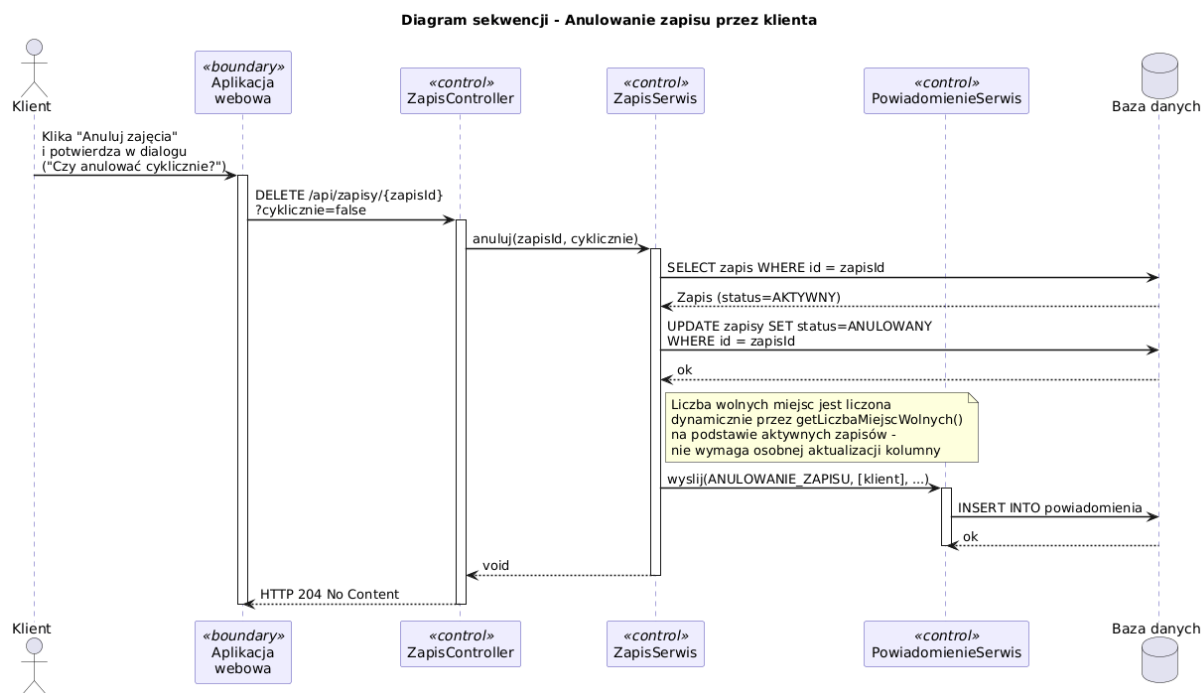
Sporządzono diagram klas dla architektury, a także odpowiednie diagramy sekwencji dla implementowanych w przyszłości funkcjonalności. Diagramy w czytelniejszej wersji można znaleźć w repozytorium na githubie w odpowiednich katalogach - pliki png oraz .puml

Diagram klas w minimalistycznej wersji w celach pokazowych (bez niektórych zależności, pełny znajduje się w repozytorium na githubie):

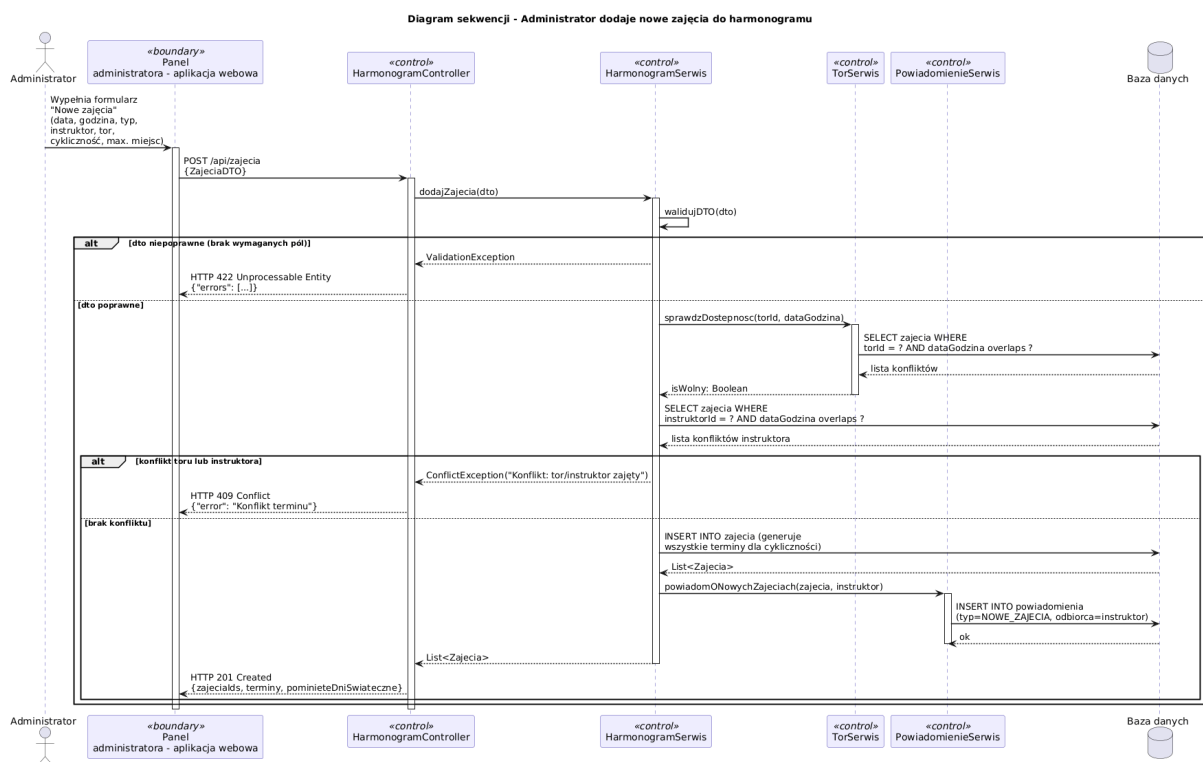


Rysunek 7: Minimalistyczny diagram klas

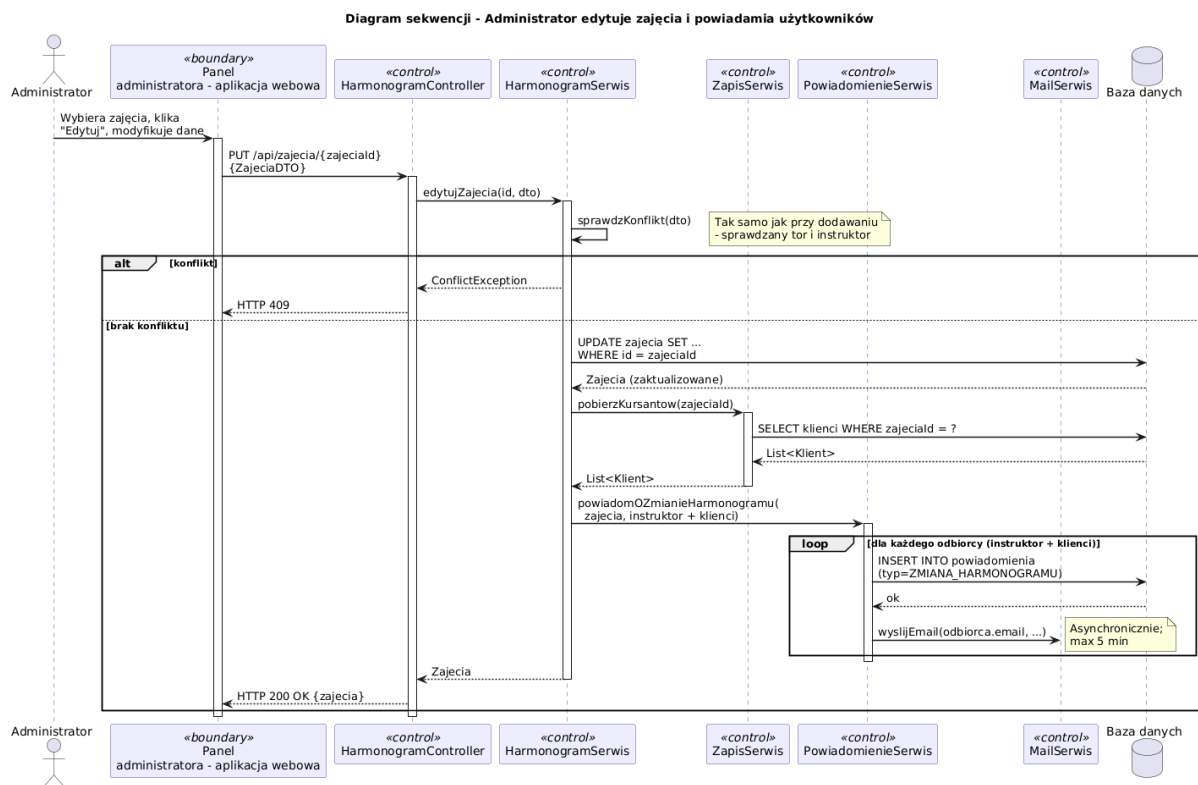
Diagramy sekwencji systemu:



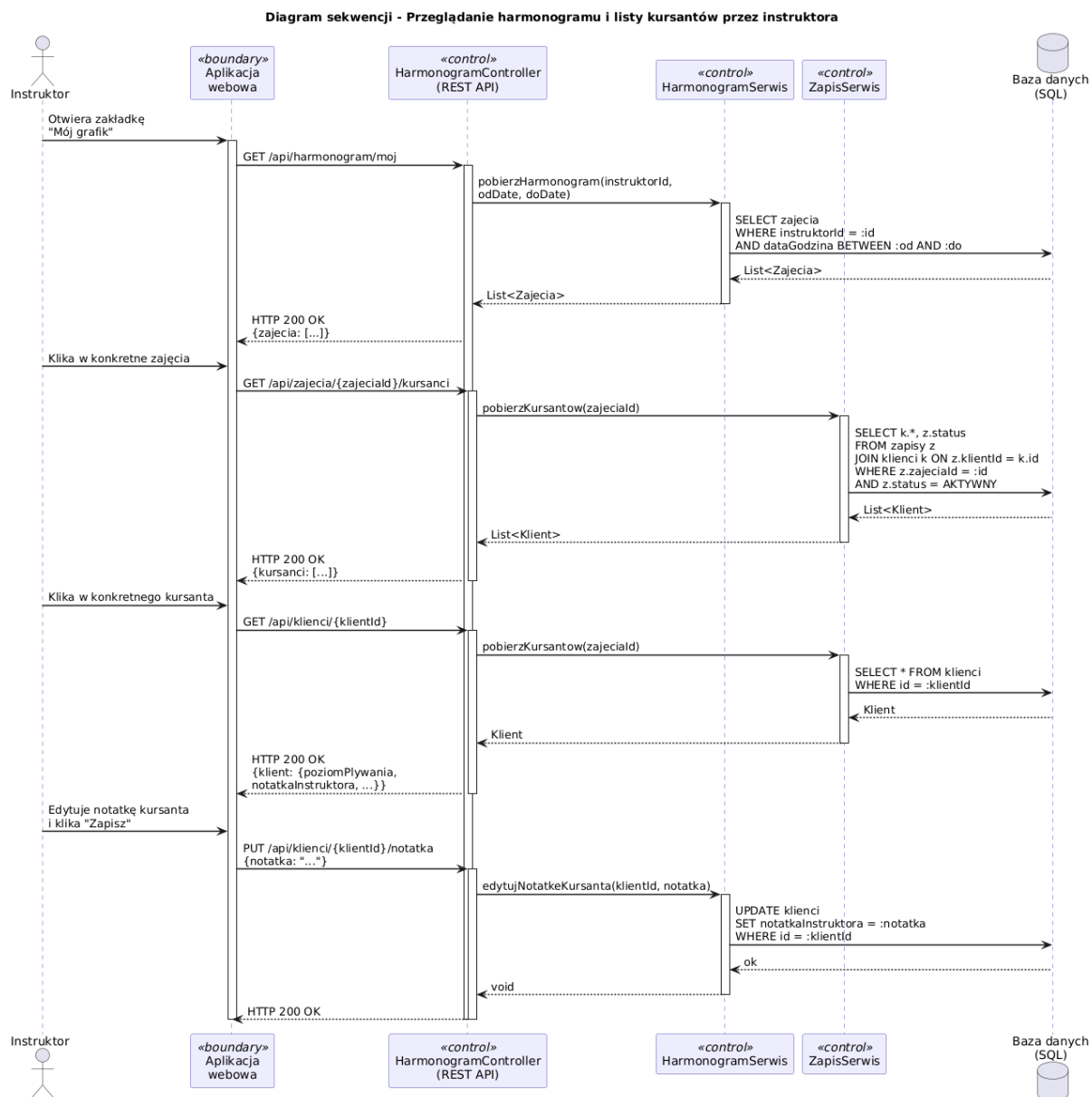
Rysunek 8: Diagram sekwencji - anulowanie zapisu na zajęcia



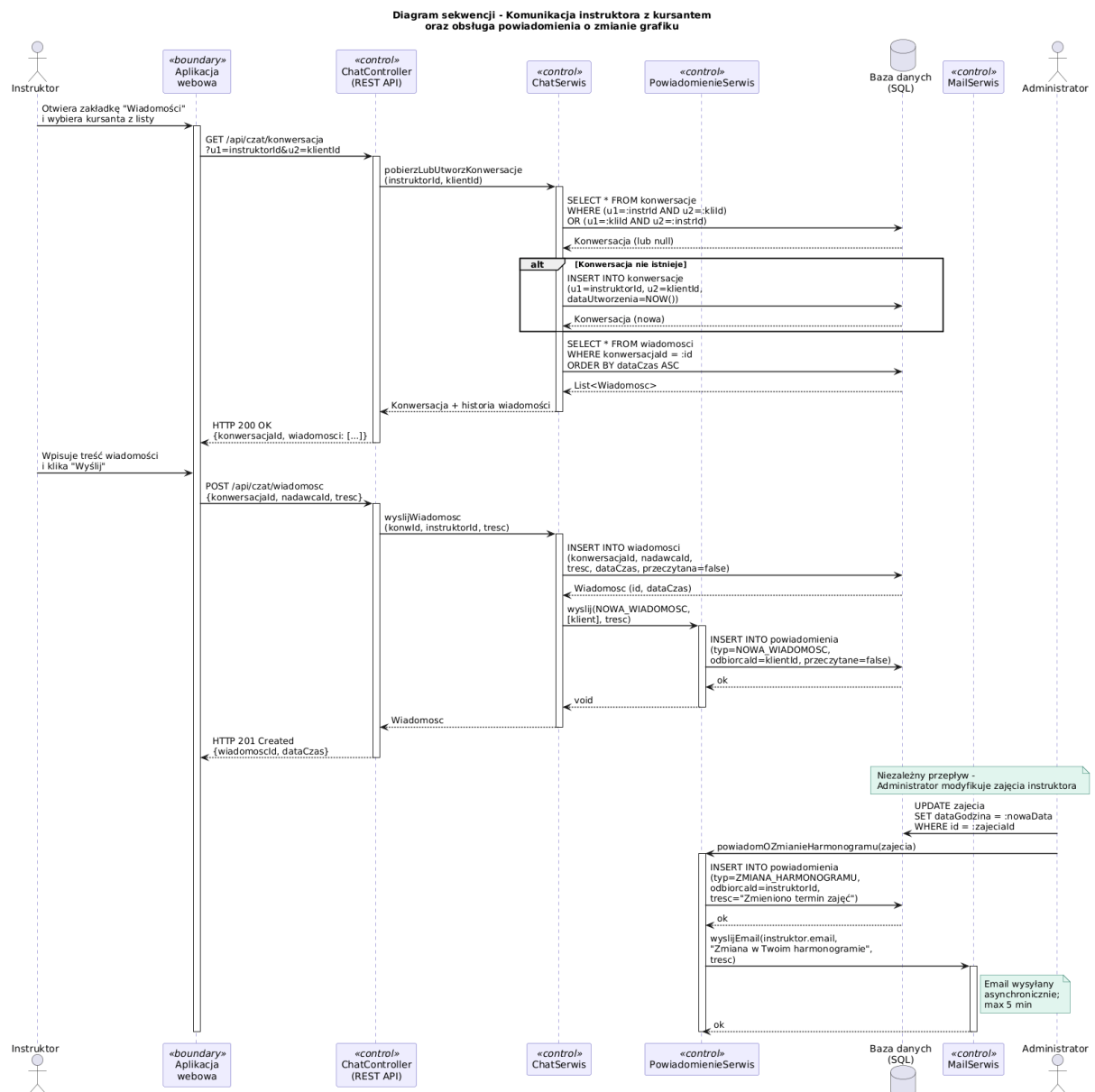
Rysunek 9: Diagram sekwencji - dodanie zajęć do harmonogramu



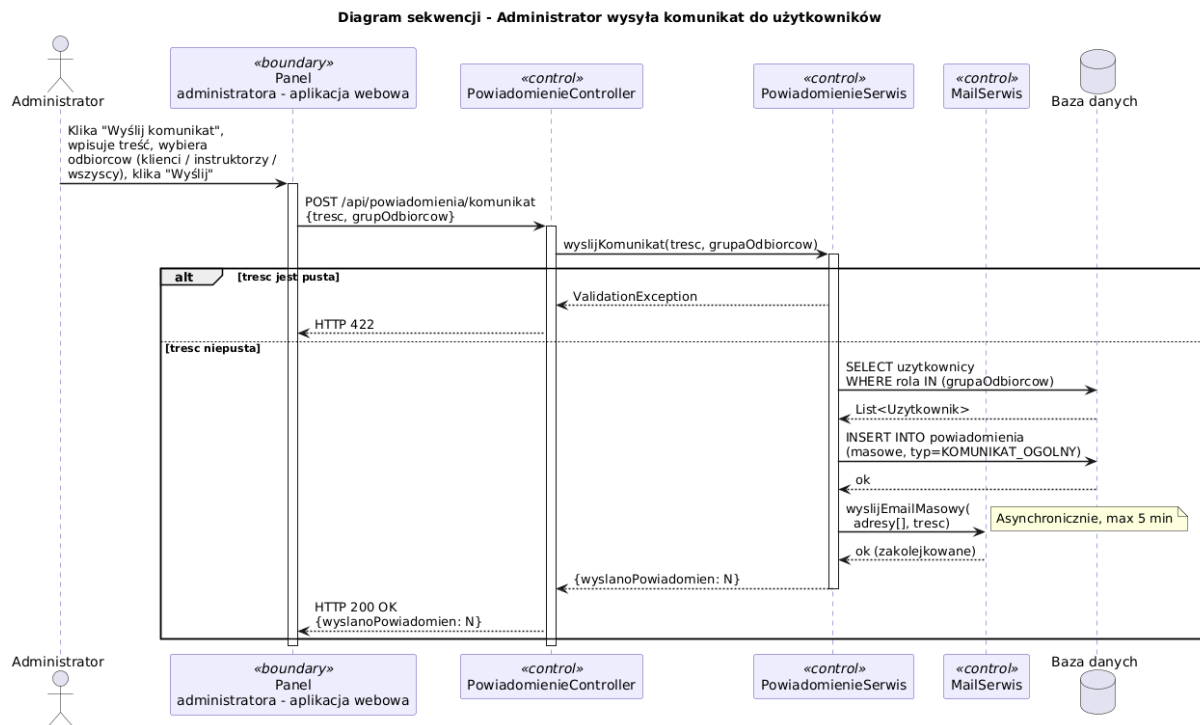
Rysunek 10: Diagram sekwencji - edycja zajęć



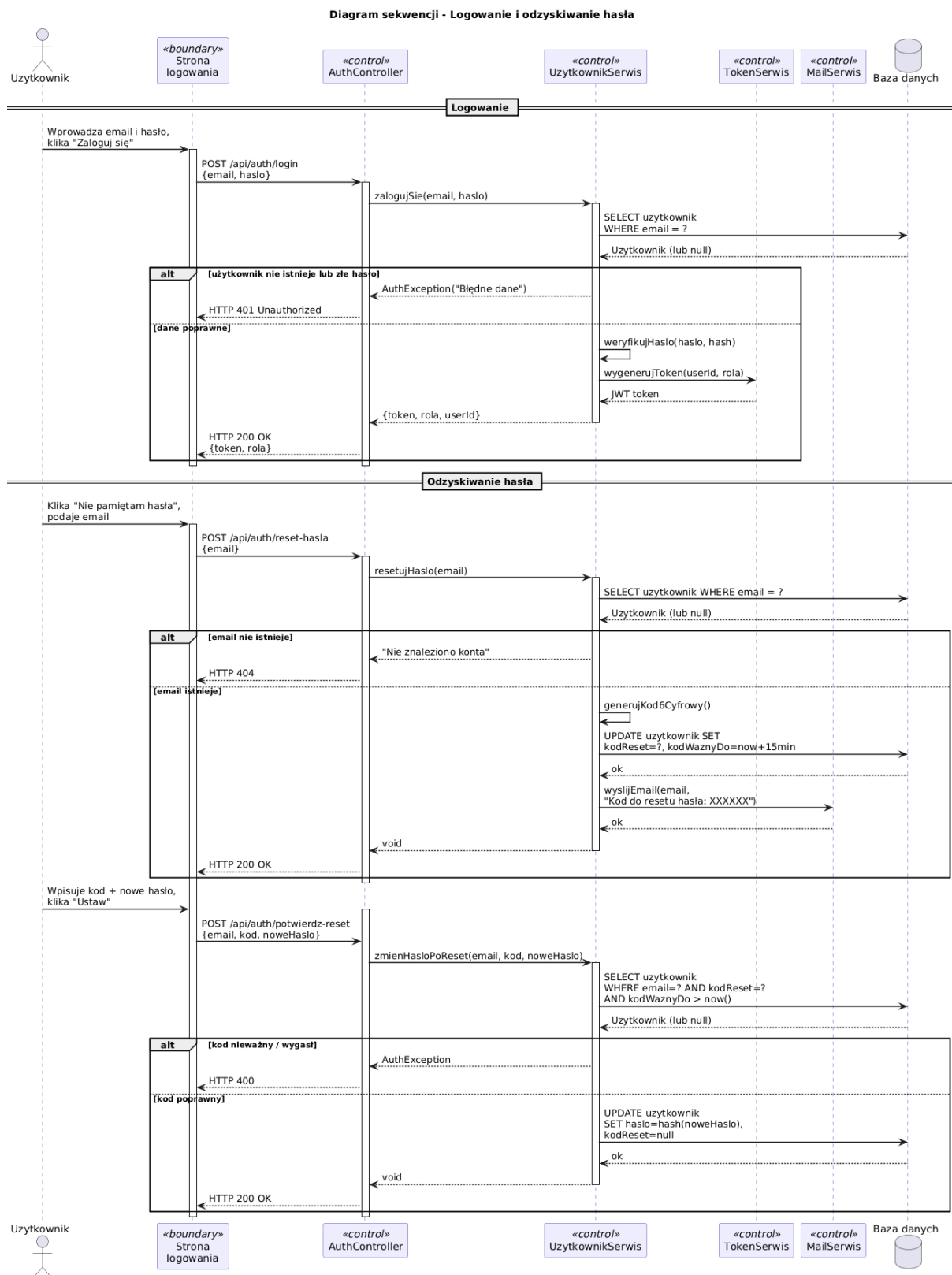
Rysunek 11: Diagram sekwencji - Harmonogram instruktora



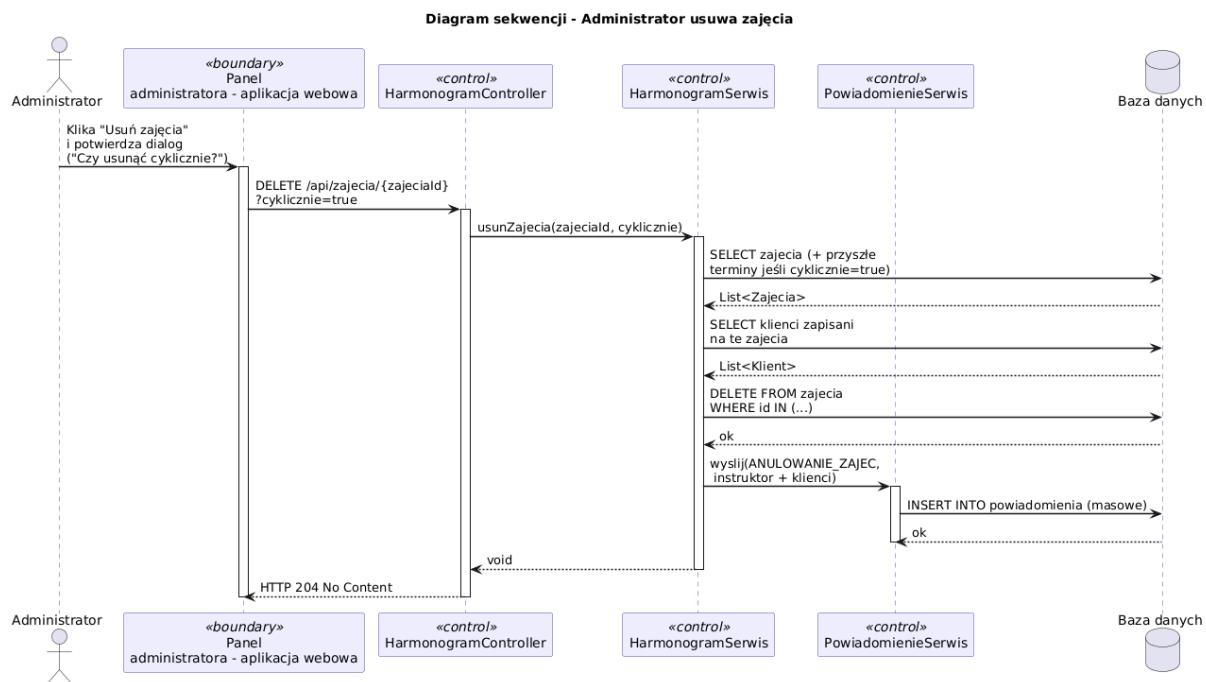
Rysunek 12: Diagram sekwencji - komunikacja i powiadomienia



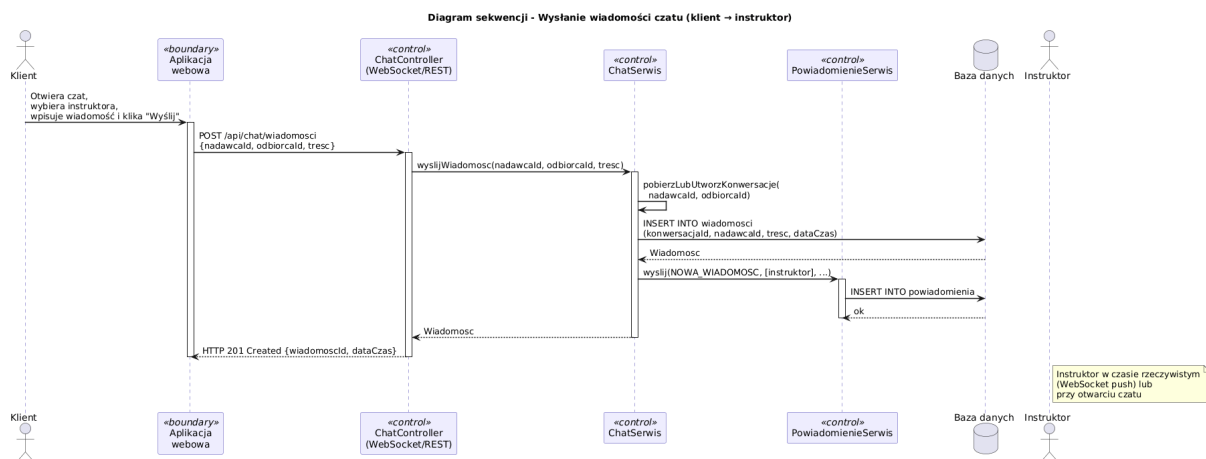
Rysunek 13: Diagram sekwencji - komunikat



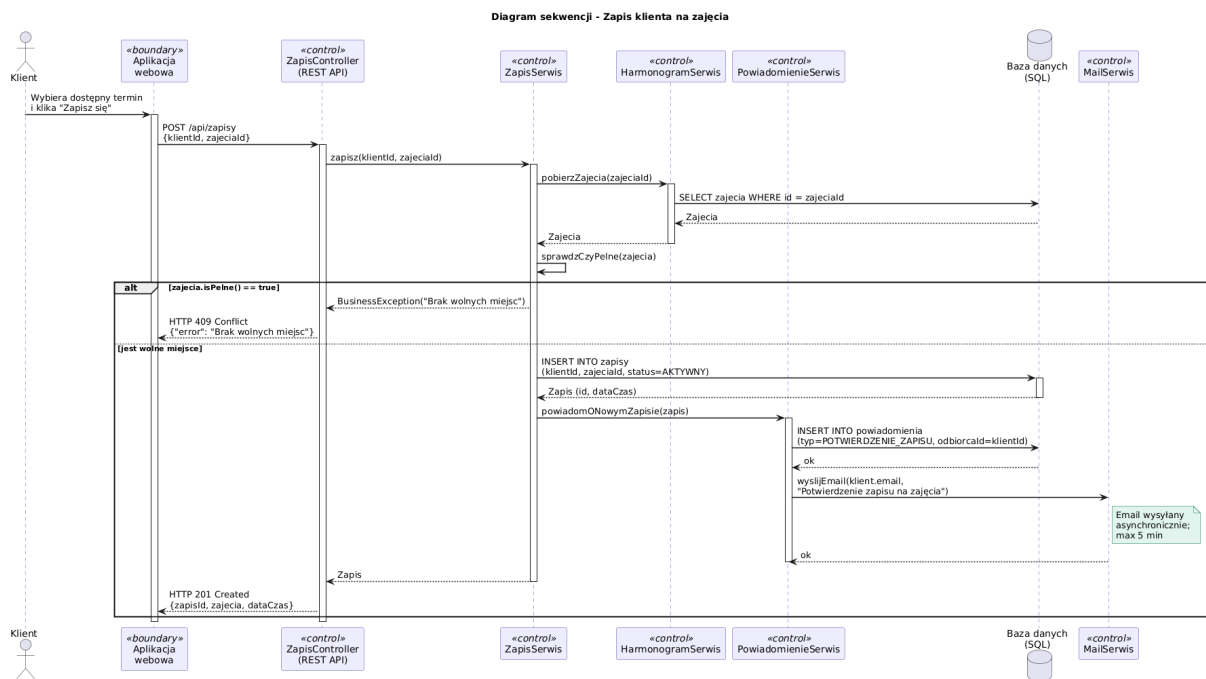
Rysunek 14: Diagram sekwencji - logowanie



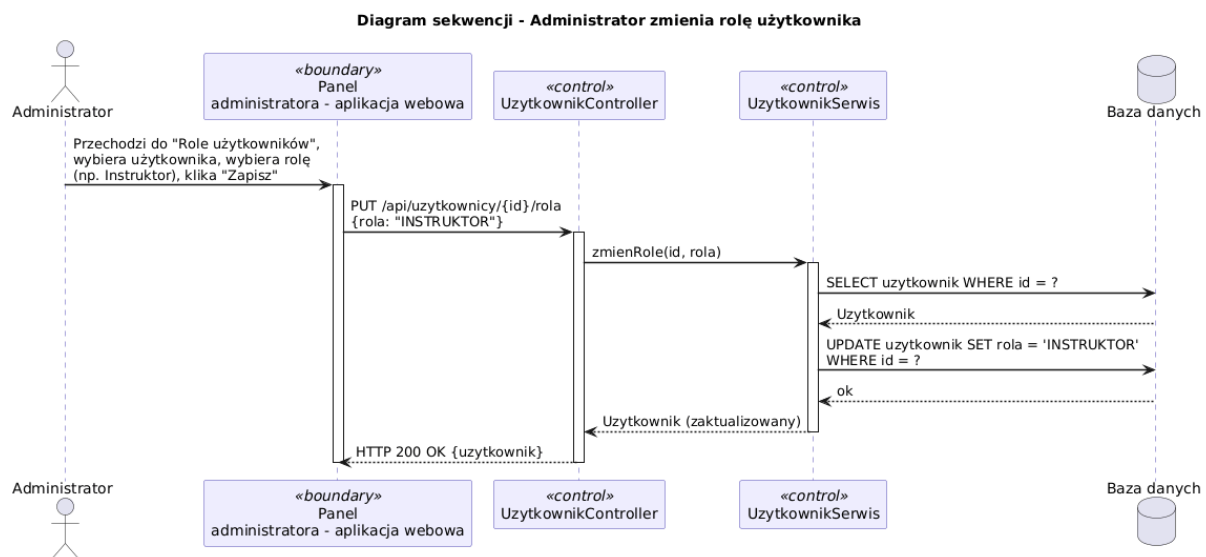
Rysunek 15: Diagram sekwencji - usuwanie zajęć



Rysunek 16: Diagram sekwencji - czat



Rysunek 17: Diagram sekwencji - zapis na zajęcia



Rysunek 18: Diagram sekwencji - zmiana roli użytkownika

10 Raport z badań interfejsu użytkownika

Projekt: Delfinek — System zarządzania szkołą pływania

Zespół: Łukasz Deja, Natalia Parszywka, Miłosz Błaszczyk

Data badania: Maj 2026

Wersja dokumentu: 1.1

10.1 Cel i zakres badania

Celem badania było zweryfikowanie użyteczności prototypu lo-fi interfejsu użytkownika systemu Delfinek przed wejściem w fazę implementacji. Badanie miało sprawdzić:

- Czy nawigacja po systemie jest intuicyjna dla wszystkich grup użytkowników?
- Czy proces zapisu na zajęcia jest zrozumiały i bezproblemowy?
- Czy ekran logowania i rejestracji jest czytelny?
- Jakie problemy napotykają użytkownicy przy pierwszym kontakcie z systemem?

10.2 Metodologia

Badanie przeprowadziliśmy sami, metodą moderowanych testów użyteczności. Pokazaliśmy interaktywny prototyp wykonany w programie Moqups **5 osobom (studentom)**.

Uczestnicy dostali do wykonania proste zadania:

Zadania jako klient:

1. Utworzenie konta i zalogowanie się
2. Wyszukanie dostępnych zajęć w nadchodzącym tygodniu
3. Zapisanie się na wybrane zajęcia
4. Sprawdzenie szczegółów zajęć (instruktor, tor, godzina)
5. Anulowanie zapisu na zajęcia
6. Skontaktowanie się z instruktorem przez czat

Zadania jako instruktor:

1. Zalogowanie się i przejrzanie grafiku
2. Wysłanie prośby o usunięcie istniejących zajęć
3. Przejrzanie i edytowanie informacji o kliencie
4. Zmiana hasła

Zadania jako pracownik biura:

1. Dodanie nowych zajęć do harmonogramu
2. Sprawdzenie dostępności torów
3. Dodanie nowych zajęć
4. Sprawdzenie szczegółów zajęć
5. Nadanie/Zmiana roli użytkownika systemu

10.3 Główne problemy i zaobserwowane błędy

Podczas sesji wyłapaliśmy kilka kluczowych problemów w interfejsie. Część z nich została otwarcie wskazana przez samych użytkowników, natomiast inne błędy (głównie projektowe i procesowe) zauważyliśmy my (twórcy) na podstawie obserwacji zachowania testerów.

Problemy krytyczne (do natychmiastowej poprawy)

- **Brak potwierdzenia zapisu:** Po kliknięciu „Zapisz się” na ekranie nic się nie pojawia. Uczestnik 4 klikał przycisk kilka razy, bo nie wiedział, czy system zadziałał.
- **Brak potwierdzenia dodania zajęć przez admina:** Uczestnik 1 zauważył, że po zatwierdzeniu formularza przez administratora i dodaniu nowej lekcji do grafiku, system nie wyświetla żadnego komunikatu potwierdzającego sukces. Użytkownik nie wie, czy operacja się powiodła bez ręcznego sprawdzania kalendarza. Dodatkowo zauważono, że przy dodawaniu cyklicznych zajęć powinna być dodana informacja o pominiętych terminach ze względu na dni wolne od pracy (świąteczne).
- **Brak informacji o godzinach zajęć w widoku torów dla pracownika biura:** Uczestnik 3 zauważył, że zajęcia były prezentowane bez wskazania czasu ich trwania, co mogło prowadzić do niejasności dotyczących harmonogramu. Dodano informację o godzinach zajęć.

Mniejsze błędy i niedogodności

- **Brak dostępu do opisu instruktora:** Podczas gdy w aktualnej wersji systemu klienci mogą już bez problemu zobaczyć opis instruktora przy logowaniu się na zajęcia (co było niemożliwe we wcześniejszej wersji), klient nie miał możliwości sprawdzenia informacji o instruktora, do którego chciał się zapisać na zajęcia.
- **Brak możliwości napisania do kursanta:** Przy przeglądaniu informacji o kliencie z perspektywy instruktora uczestnik 3 zauważył, że brakuje przydatnej funkcji napisania do swojego kursanta, bez konieczności wybierania z listy wszystkich użytkowników systemu.
- **Brak informacji o cyklicznej rezygnacji z zajęć:** (zauważone przez twórców) Podczas usuwania lub anulowania uczestnictwa w zajęciach dostępna jest opcja cyklicznej rezygnacji. Interfejs nie informuje jednak, których konkretnie terminów dotyczy ta operacja. Aby sprawdzić, z jakich zajęć użytkownik rezygnuje, konieczne jest samodzielne przejście harmonogramu.

10.4 Podsumowanie spostrzeżeń uczestników

Użytkownicy bez większych problemów wykonywali większość zadań i uznali podstawową nawigację systemu za intuicyjną. Podczas testów użytkownicy zwrócili jednak uwagę na niedociągnięcia w obecnym projekcie:

- Najczęściej zgłaszanym problemem był brak jasnej informacji zwrotnej po wykonaniu operacji (np. zapis na zajęcia, dodanie zajęć).
- Użytkownicy oczekiwali większej przejrzystości i dostępności informacji kontekstowych (np. opis instruktora, godziny zajęć).

- Problemy wynikały głównie z braku komunikatów systemowych, a nie z samej struktury interfejsu.
- Pomimo uwag, ogólny odbiór prototypu był pozytywny.

10.5 Rekomendacje zmian

Na podstawie uwag użytkowników oraz naszych własnych obserwacji wprowadzono następujące poprawki do prototypu:

- Dodanie komunikatów potwierdzających wykonanie kluczowych operacji, takich jak zapis na zajęcia, anulowanie uczestnictwa czy dodanie nowych zajęć do harmonogramu.
- Rozszerzenie informacji dostępnych dla klientów poprzez umożliwienie podglądu opisu instruktora przed zapisaniem się na zajęcia.
- Uzupełnienie widoku torów o godziny rozpoczęcia i zakończenia zajęć w celu zwiększenia czytelności harmonogramu.
- Dodanie możliwości szybkiego kontaktu instruktora z kursantem bez konieczności wyszukiwania go na liście wszystkich użytkowników.
- Wyświetlanie informacji o terminach objętych cykliczną rezygnacją z zajęć przed zatwierdzeniem operacji.
- Dodanie informacji o dodawanych terminach przez administratora jak i pominiętych podczas tworzenia harmonogramów cyklicznych obejmujących dni wolne od pracy i święta.

10.6 Wnioski

Przeprowadzone badanie z udziałem 5 użytkowników dostarczyło kluczowych wniosków, które pozwoliły zoptymalizować system Delfinek przed etapem wdrożenia:

- System w podstawowym zakresie okazał się intuicyjny i zrozumiały dla większości użytkowników, którzy bez większych trudności realizowali przygotowane scenariusze zadań.
- Największym problemem był brak informacji zwrotnej po wykonaniu operacji, co prowadziło do niepewności użytkowników co do skuteczności wykonanych działań.
- Istotne braki dotyczyły również dostępności informacji kontekstowych, takich jak szczegóły dotyczące instruktora, godziny zajęć czy zakres wykonywanych operacji, co utrudniało podejmowanie decyzji w systemie.
- Pomimo wskazanych niedociągnięć, ogólny odbiór prototypu był pozytywny, a struktura systemu została oceniona jako przejrzysta.

11 Implementacja projektu

Zgodnie z pierwszym paragrafem punktu 6, w obecnej wersji projektu, nie zaimplementowano pełnej wersji produkcyjnej projektu. Frontend pozostawiono na etapie prototypu interfejsu. Backend zaimplementowano w uproszczonej formie - bez rzeczywistej bazy danych, jedynie z "bazą danych" w pamięci komputera, a co za tym również idzie bez przesyłania requestów poprzez REST API. Dla prostoty i uniknięcia niepotrzebnych komplikacji w tej uproszczonej wersji implementacyjnej nie zastosowano frameworka Spring Boot, kod napisano w Javie, testowano na wersji Java 21.

11.1 Logika backendu

Zaimplementowany backend to aplikacja napisana w czystej Javie 21 (na innej projekt może nie działać, testy jednostkowe nie działają!), bez użycia frameworków (Spring Boot, Hibernate itp.) ani rzeczywistej bazy danych. Zamiast bazy danych dane przechowywane są w pamięci aplikacji (tzw. *in-memory*). Cały projekt jest zorganizowany w kilka pakietów, opisanych w kolejnych podpunktach.

Pakiet model

Zawiera klasy reprezentujące obiekty dziedziny systemu. Główna klasa bazowa to **Uzytkownik**, po której dziedziczą trzy klasy: **Klient**, **Instruktor** oraz **Administrator**. Każdy użytkownik posiada identyfikator, imię, nazwisko, adres e-mail, hasło, numer telefonu i datę urodzenia. Klasa **Klient** przechowuje dodatkowo poziom zaawansowania pływania oraz listę powiadomień. Klasa **Instruktor** zawiera opis.

Pozostałe klasy modelu:

- **Zajecia** – reprezentuje jeden termin zajęć w harmonogramie (data, godzina, czas trwania, typ zajęć, maksymalna liczba miejsc, cykliczność, przypisany instruktor i tor). Liczba wolnych miejsc nie jest polem w klasie – jest wyliczana dynamicznie na podstawie listy aktualnych uczestników metodą `getLiczbaMiejscWolnych()`.
- **Zapis** – powiązanie klienta z zajęciami; przechowuje status (aktywny lub anulowany).
- **Tor** – tor basenowy ze statusem (wolny, zajęty, remont) oraz metodą sprawdzającą dostępność w danym przedziale czasowym.
- **Konwersacja i Wiadomosc** – obsługa czatu; konwersacja posiada dwóch uczestników i listę wiadomości.
- **Powiadomienie** – powiadomienie dla użytkownika z typem, tytułem i treścią.

Typy wyliczeniowe - zgodne z diagramem klas.

Pakiet dto

Zawiera klasy DTO (Data Transfer Object) – czyli obiekty do przekazywania danych wejściowych do serwisów:

- **RegistrationDTO** – dane rejestracji nowego klienta (imię, nazwisko, e-mail, hasło, telefon, data urodzenia).

- **ZajeciaDTO** – dane potrzebne do utworzenia lub edycji zajęć (data i godzina, czas trwania, typ zajęć, id instruktora, id toru, liczba miejsc, uwagi, cykliczność, data końca cyklu).

Pakiet `exception`

Zawiera własne wyjątki domenowe zamiast ogólnych wyjątków Javy:

- **ValidationException** – niepoprawne dane wejściowe (np. błędny format hasła, brakujące pole).
- **ConflictException** – konflikt biznesowy (np. tor lub instruktor już zajęty w danym terminie).
- **NotFoundException** – nie znaleziono obiektu o podanym id.
- **BusinessException** – ogólny błąd logiki biznesowej (np. brak wolnych miejsc na zajęciach).

Pakiet `repository`

Repozytoria symulują warstwę dostępu do danych. Klasa abstrakcyjna **InMemoryRepository<T>** dostarcza wspólną implementację operacji CRUD – przechowuje obiekty w mapie (**ConcurrentHashMap**) i automatycznie nadaje im kolejne identyfikatory (**AtomicLong**). Każde konkretne repozytorium rozszerza tę klasę i może dodawać własne metody wyszukiwania.

Zaimplementowane repozytoria:

- **UzytkownikRepository** – wyszukiwanie po e-mailu i po roli.
- **ZajeciaRepository** – wyszukiwanie po przedziale dat, po instruktorze, a także wykrywanie konfliktów terminu toru i instruktora.
- **ZapisRepository** – wyszukiwanie zapisów po kliencie i pobieranie aktywnych uczestników zajęć.
- **TorRepository** – wyszukiwanie torów po statusie.
- **PowiadomienieRepository** – wyszukiwanie powiadomień po odbiorcy.
- **KonwersacjaRepository** – wyszukiwanie konwersacji pomiędzy dwoma konkretnymi użytkownikami.

Pakiet `service` i `service.impl`

Serwisy zawierają całą logikę biznesową aplikacji. Każdy serwis jest zdefiniowany jako interfejs w pakiecie `service`, a jego implementacja znajduje się w pakiecie `service.impl`. Zaimplementowano następujące serwisy:

- **HarmonogramSerwis / HarmonogramSerwisImpl** – zarządzanie harmonogramem zajęć: dodawanie (z obsługą zajęć cyklicznych), edycja (ze sprawdzaniem konfliktów terminu z wyłączeniem edytowanych zajęć), usuwanie (opcjonalnie cykliczne – usuwa wszystkie przyszłe wystąpienia danego cyklu), pobieranie harmonogramu w zadanym przedziale dat, przydział instruktora lub toru do istniejących zajęć.

- **ZapisSerwis / ZapisSerwisImpl** – obsługa zapisów klientów na zajęcia: zapis klienta (ze sprawdzeniem dostępności miejsc i powiadomieniem instruktora), anulowanie zapisu (opcjonalnie cykliczne – anuluje zapisy na wszystkie przyszłe zajęcia tego samego cyklu), pobieranie zapisów klienta i listy kursantów.
- **UzytkownikSerwis / UzytkownikSerwisImpl** – rejestracja nowego klienta z walidacją danych (format e-mail, długość numeru telefonu, złożoność hasła: minimum 6 znaków, wielka litera, cyfra i znak specjalny), zmiana roli użytkownika, obsługa resetu hasła przez kod jednorazowy przechowywany w mapie w pamięci.
- **ChatSerwis / ChatSerwisImpl** – obsługa czatu: pobranie istniejącej lub utworzenie nowej konwersacji między dwoma użytkownikami, wysyłanie wiadomości (z powiadomieniem drugiego uczestnika), pobieranie historii wiadomości.
- **PowiadomienieSerwis / PowiadomienieSerwisImpl** – wysyłanie powiadomień do odbiorców: po zapisie klienta, po anulowaniu zapisu, po zmianie harmonogramu, po pojawieniu się nowego kursanta, oraz masowe komunikaty administratora do wybranej grupy ról (np. wszyscy klienci). Każde powiadomienie jest zapisywane w repozytorium i dołączane do obiektu odbiorcy. Dla wybranych zdarzeń (np. zmiana harmonogramu, potwierdzenie zapisu) wywoływana jest symulowana wysyłka e-maila przez **MailSerwis**.
- **TorSerwis / TorSerwisImpl** – zarządzanie torami basenowymi: sprawdzanie dostępności toru w podanym przedziale czasu, zgłaszanie i kończenie remontu (zmiana statusu toru).
- **MailSerwis / MailSerwisImpl** – symulowana wysyłka e-maili: zamiast rzeczywistego serwera SMTP, metody jedynie wypisują treść wiadomości na konsolę. Interfejs jest przygotowany pod podłączenie prawdziwego **JavaMailSender** w przyszłości.
- **TokenSerwis / TokenSerwisImpl** – generowanie i weryfikacja tokenów (w uproszczonej wersji jako losowe ciągi znaków), używanych przy resecie hasła.

Klasa Main

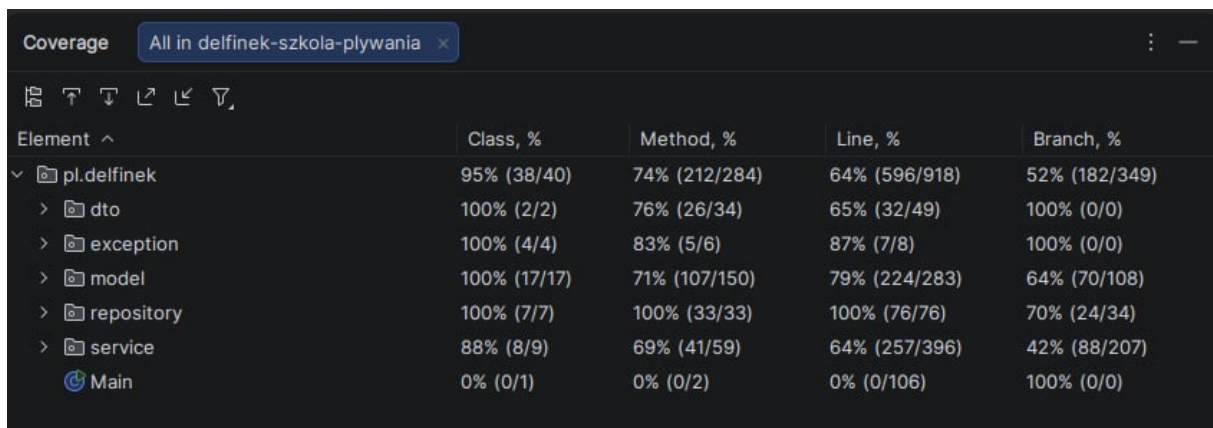
Klasa **Main** pełni rolę demonstracyjną – prezentuje kolejno działanie całego systemu bez interfejsu użytkownika ani REST API. Ręcznie tworzy wszystkie repozytoria i serwisy (wstrzykując zależności przez konstruktory), a następnie wykonuje scenariusz testowy obejmujący:

1. Inicjalizację danych: dodanie administratora, instruktora i dwóch torów.
2. Rejestrację klientów (w tym próbę rejestracji z niepoprawnym hasłem, która kończy się wyjątkiem **ValidationException**).
3. Dodanie serii zajęć cyklicznych (co tydzień przez cztery tygodnie) oraz próbę dodania kolidujących zajęć na tym samym torze, zakończoną wyjątkiem **ConflictException**.
4. Zapis dwóch klientów na pierwsze zajęcia i wyświetlenie liczby wolnych miejsc.
5. Anulowanie zapisu jednego klienta i weryfikację aktualizacji liczby miejsc.
6. Edycję godziny zajęć i wyświetlenie powiadomień wysłanych do uczestnika.

7. Wymianę wiadomości przez czat między instruktorem a klientem.
8. Wysłanie komunikatu masowego do wszystkich klientów przez administratora.
9. Zgłoszenie i zakończenie remontu toru.
10. Cykliczne usunięcie wszystkich przyszłych zajęć z cyklu.

Testy jednostkowe

W katalogu testowym (`src/test/java`) umieszczono klasy testowe dla wszystkich zaimplementowanych backendowych klas. Zaimplementowano testy jednostkowe z Mockowaniem, testujące najważniejsze funkcjonalności i zależności projektu. Łącznie zaimplementowano 107 testów jednostkowych. Testy należy uruchamiać na Javie 21 lub 17!:



The screenshot shows the 'Coverage' tool in IntelliJ IDEA. The title bar indicates 'All in delfinek-szkola-plywania'. The table below displays the coverage statistics for various elements of the project.

Element ^	Class, %	Method, %	Line, %	Branch, %
pl.delfinek	95% (38/40)	74% (212/284)	64% (596/918)	52% (182/349)
dto	100% (2/2)	76% (26/34)	65% (32/49)	100% (0/0)
exception	100% (4/4)	83% (5/6)	87% (7/8)	100% (0/0)
model	100% (17/17)	71% (107/150)	79% (224/283)	64% (70/108)
repository	100% (7/7)	100% (33/33)	100% (76/76)	70% (24/34)
service	88% (8/9)	69% (41/59)	64% (257/396)	42% (88/207)
Main	0% (0/1)	0% (0/2)	0% (0/106)	100% (0/0)

Rysunek 19: Pokrycie testami projektu

Uzyskano satysfakcjonujące pokrycie testami, uwzględniając to, że nie testowano Maina, prostych metod takich jak niektóre gettery, settery, konstruktory czy metody pomocnicze, w serwisach rozdzielono również prototypy metod - interfejsy, od implementacji - klasy dziedziczące, co również powoduje fałszywe zniekształcenie pokrycia testowego, fałszywe zaniżenie - nie testowano interfejsów.